Community Experience Distilled

Corona SDK Application Design

A quick and easy guide to creating your very own mobile apps with Corona SDK

Daniel Williams

[PACKT] PUBLISHING

www.allitebooks.com

Corona SDK Application Design

A quick and easy guide to creating your very own mobile apps with Corona SDK

Daniel Williams

[PACKT]
PUBLISHING

BIRMINGHAM - MUMBAI

www.allitebooks.com

Corona SDK Application Design

Copyright © 2013 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: June 2013

Production Reference: 1060613

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-84969-736-1

www.packtpub.com

Cover Image by Suresh Mogre (suresh.mogre.99@gmail.com)

www.allitebooks.com

Credits

Author
Daniel Williams

Reviewers
Mitch Allen
Predrag Končar
Jamie Trinder

Acquisition Editor
Joanne Fitzpatrick

Commissioning Editor
Sruthi Kutty

Technical Editors
Prasad Dalvi
Pushpak Poddar
Saumya Kunder

Project Coordinator
Siddhant Shetty

Proofreader
Paul Hindle

Indexer
Hemangini Bari

Production Coordinator
Nitesh Thakur

Cover Work
Nitesh Thakur

www.allitebooks.com

About the Author

Daniel Williams is a web developer, mobile game developer, online specialist, and founder of That's So Panda (`www.thatssopanda.com`). Since starting his own mobile adventure with Corona SDK a couple of years ago, Daniel has released several app titles and has written several tutorials on `www.thatssopanda.com` for individuals just starting with Corona SDK. When he's not busy reading about the latest technology news or working on his latest app title, he can be found watching the latest movies, discovering new local restaurants, or playing video games.

I would like to thank my wife, for always being my personal cheerleader.

---

www.allitebooks.com

About the Reviewers

Mitch Allen has worked on software projects for Lotus, IBM, Dragon, Nuance, Yahoo, and Intuit, as well as for a few startups. He specializes in everything from Enterprise Web Apps to Mobile. He has worked in Silicon Valley, Silicon Alley, and along Boston's Route 128. He currently resides in Western Pennsylvania. Previously, he has reviewed the book *Corona SDK Mobile Game Development Beginner's Guide* by *Packt Publishing*.

For more information, please visit his website at `http://mitchallen.com` where he blogs about everything from mobile game development to robotics.

Predrag Končar is a founder of Glyphed – a multidisciplinary research studio. His primary areas of interest are programming and combining technology and art. Predrag is also into robotics, image and audio processing, and interactive design, and he likes to spend his free time painting. In the last 12 years, he has worked as a Technical and Creative Director on many online projects, published over 40 online games, participated in the production of several iOS apps, and worked with Packt Publishing as a technical reviewer on the book *Corona SDK Mobile Game Development Beginner's Guide*. He has a strong background in ActionScript, Lua, MEL script, Maya, and Python. Predrag is also a member of MENSA and ACM SIGGRAPH.

Jamie Trinder is the Technical Director for T and G Apps Ltd, who create mobile applications and games for multiple mobile platforms. Jamie has a background with various coding languages and has been a part of creating numerous mobile applications. Jamie uses Corona and Lua extensively from day to day and finds them invaluable tools for rapidly prototyping ideas for the company.

www.allitebooks.com

www.PacktPub.com

Support files, eBooks, discount offers and more

You might want to visit `www.PacktPub.com` for support files and downloads related to your book.

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at `www.PacktPub.com` and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at `service@packtpub.com` for more details.

At `www.PacktPub.com`, you can also read a collection of free technical articles, sign up for a range of free newsletters and receive exclusive discounts and offers on Packt books and eBooks.

`http://PacktLib.PacktPub.com`

Do you need instant solutions to your IT questions? PacktLib is Packt's online digital book library. Here, you can access, read and search across Packt's entire library of books.

Why Subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print and bookmark content
- On demand and accessible via web browser

Free Access for Packt account holders

If you have an account with Packt at `www.PacktPub.com`, you can use this to access PacktLib today and view nine entirely free books. Simply use your login credentials for immediate access.

www.allitebooks.com

Table of Contents

Preface **1**

Chapter 1: Getting Started with Your Mobile Adventure **5**

What is Corona SDK? **5**
Prerequisites for Corona SDK **6**
Downloading, installing, and using Corona SDK **7**
Installing for Mac OS X 7
Installing for Microsoft Windows 8
Using Corona SDK 8
Summary **10**

Chapter 2: Learning Lua **11**

Using variables with Lua **11**
Using the terminal **12**
Expressions and operators **13**
Arithmetic expressions 13
Comparison operators 13
Making decisions **14**
The if-then statement 14
The if-then-else statement 15
Loops, loops, and loops **15**
The for loop 16
The while loop 17
More flexibility with tables 17
Using functions 19
Summary **20**

www.allitebooks.com

Chapter 3: Creating Your First App **21**
Using a text editor **22**
Notepad++ 23
Sublime Text 23
Corona-specific editors 23
Creating a new project **23**
An introduction to Storyboard 26
The createScene function 27
The enterScene function 27
The exitScene function 28
The destroyScene function 28
Switching scenes 28
Destroying scenes 28
Using widgets **29**
Starting with main.lua **31**
view1.lua – about us 35
view2.lua – your BMI 38
view3.lua – contact us 46
config.lua – set up dynamic content resolution 49
App icons **50**
Summary **51**
Chapter 4: Creating a Puzzle App **53**
A quick introduction to puzzle games **53**
Downloading the starter kit 54
My Jigsaw Puzzle **54**
build.settings 55
config.lua 56
main.lua 56
menu.lua 56
gameplay.lua 59
Getting local libraries 59
Creating variables 59
Creating display groups 61
The shuffle function 61
Creating the shuffle function 62
Dragging-and-dropping display objects **62**
Creating the drag function 64
Summary **72**

www.allitebooks.com

Chapter 5: Testing and Publishing **73**
How to test your app **73**
iOS 73
Android 74
Publishing your app **74**
Publishing to Apple's App Store 74
Publishing to Google Play 75
Other resources and the Corona community **75**
Appendix: Tools, Services, and Marketing **77**
Third-party tools **77**
Third-party services **78**
Marketing tips **78**
Index **81**

www.allitebooks.com

Preface

This book is designed to teach you the core concepts of the Lua programming language to get you started with building your own apps quickly and easily. After you've learned some of the core concepts, you will learn how to build your first app and your first game using the Corona SDK product. You will also discover how to get your app on the App Store or Google Play.

What this book covers

Chapter 1, *Getting Started with Your Mobile Adventure*, will get you acquainted with the Corona SDK product and will guide you through the setup of Corona SDK so that you can start creating apps in no time.

Chapter 2, *Learning Lua*, teaches you the core concepts of the Lua language. We will learn about the basics of variables, tables, decision statements, and functions. Everything in this chapter is designed to jump-start your app development adventure.

Chapter 3, *Creating Your First App*, will gently guide you along the path of creating your first app. You'll learn about what text editors to use, how to take advantage of the features in Corona SDK, and how to create a real-life example app (no "Hello World!" here).

Chapter 4, *Creating a Puzzle App*, continues your app development adventure by teaching you how to create your very first game. You'll learn how to use drag-and-drop, widgets, and events, all while creating a puzzle app that's playable by all ages.

Chapter 5, *Testing and Publishing*, discusses how to start testing your app. Once you've learned how to test your app, this chapter will explain how to publish your app on the App Store or Google Play.

Appendix, *Tools, Services, and Marketing*, provides information on how to get the most out of the Corona SDK product. Inside this appendix, you will discover third-party tools and services that will help speed up your development as well as marketing tips to help make your app successful.

What you need for this book

You will need a couple of items before you can develop apps with Corona SDK:

- Mac OS X 10.7 or later, Intel Mac
- Windows XP or later, 1 GHz processor
- Java 6 SDK
- A text editor such as Notepad++ or Sublime Text 2
- The latest public build of Corona SDK – Build 2012.971

If you would like to publish your apps, you will need a Google Play for Android or an iTunes developer account for iOS.

Who this book is for

This book is for anyone who is interested in creating applications for Android or iOS. You do not need previous development or programming experience.

Conventions

In this book, you will find a number of styles of text that distinguish between different kinds of information. Here are some examples of these styles, and an explanation of their meaning.

Code words in text are shown as follows: "To install Corona, open the `.dmg` file and follow the on-screen instructions."

A block of code is set as follows:

```
if(age < 29) then
print('You are not quite 30!');
end
```

New terms and **important words** are shown in bold. Words that you see on the screen, in menus or dialog boxes for example, appear in the text like this: "Once installed, you will be presented with the **Corona Developer Login** screen."

Warnings or important notes appear in a box like this.

Tips and tricks appear like this.

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book—what you liked or may have disliked. Reader feedback is important for us to develop titles that you really get the most out of.

To send us general feedback, simply send an e-mail to `feedback@packtpub.com`, and mention the book title via the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide on `www.packtpub.com/authors`.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at `http://www.packtpub.com`. If you purchased this book elsewhere, you can visit `http://www.packtpub.com/support` and register to have the files e-mailed directly to you.

Downloading the color images of this book

We also provide you a PDF file that has color images of the screenshots/diagrams used in this book. The color images will help you better understand the changes in the output. You can download this file from `http://www.packtpub.com/sites/default/files/downloads/7361OT_ColoredImages.pdf`.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books—maybe a mistake in the text or the code—we would be grateful if you would report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting `http://www.packtpub.com/submit-errata`, selecting your book, clicking on the **errata submission form** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded on our website, or added to any list of existing errata, under the Errata section of that title. Any existing errata can be viewed by selecting your title from `http://www.packtpub.com/support`.

Piracy

Piracy of copyrighted material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at `copyright@packtpub.com` with a link to the suspected pirated material.

We appreciate your help in protecting our authors, and our ability to bring you valuable content.

Questions

You can contact us at `questions@packtpub.com` if you are having a problem with any aspect of the book, and we will do our best to address it.

1
Getting Started with Your Mobile Adventure

Welcome to getting started with your mobile adventure! If you are interested in building mobile apps, then you've come to the right place. Whether you are interested in building the next Angry Birds, building an educational app that helps children learn while playing, or building an app for a local business, this book will help you from start to finish. In this first chapter, we will be covering the following topics:

- What is Corona SDK?
- The prerequisites for Corona SDK
- Downloading, installing, and using Corona SDK

If you are already working with Corona SDK and have experience with the Lua programming language, feel free to skip to *Chapter 3*, *Creating Your First App*. For everyone else, let's get started!

What is Corona SDK?

Corona SDK is a mobile development platform that allows you to create cross-browser games, apps, and e-books. This platform utilizes a scripting language called Lua, which is an easy-to-learn programming language.

Not only will Corona SDK help you develop your app faster than by using traditional means, but you will also be able to easily incorporate Box2D physics, Game Center, sprite sheets, social media, and a whole lot more into your app. You can also monetize your apps with advertising options and in-app purchases.

At the time of writing this book, Corona SDK offers a free and paid structure of their product. The free structure, known as Starter, allows you to build and publish apps to iOS, Android, Kindle, and the Nook. However, you will not have access to premium features such as in-app purchases unless you upgrade to the paid product, known as Pro. The Pro version also allows you to download the latest Daily Builds.

Corona Labs, the company behind Corona SDK, is continuously adding new features to their mobile development platform. Even as this book is being written, Corona Labs is adding more features to give you a better product.

Prerequisites for Corona SDK

The Corona SDK product is available for Mac OS X and Microsoft Windows. Just like any software product, there are system requirements when running the product, as well as slight differences depending on which operating system you use.

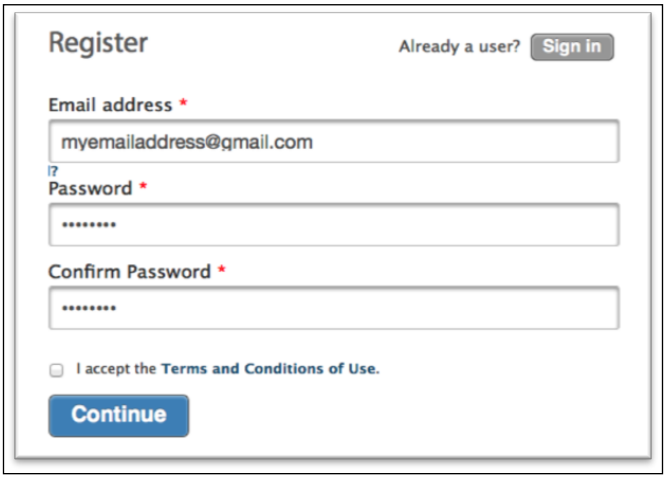
- Mac OS X
  - Mac OS X 10.7 or later
  - Intel Mac
  - Allows you to publish for iOS 4.5 or greater and Android OS 2.2 or greater (with ARMv7)
- Microsoft Windows
  - Windows XP or later
  - Minimum of 1 GHz processor
  - Only capable of publishing for Android OS 2.2 or greater (with ARMv7)

With Microsoft Windows, you are not able to publish for iOS due to restrictions placed by Apple.

Both products include the Corona SDK simulator, sample code, and a demo account for an affiliate product. You will also need a text editor such as Notepad++ and Sublime Text 2 to edit the source code of your app. However, we will be learning more about text editors in *Chapter 3*, *Creating Your First App*.

Downloading, installing, and using Corona SDK

Our first real step in our mobile adventure is to head over to `www.coronalabs.com` and download the free trial product. Once you are on this page, you will have to register for an account with Corona Labs.



After you register, you will have the option to download for Mac OS X or Microsoft Windows. Choose the appropriate operating system and click on the download link.

Installing for Mac OS X

If you've downloaded Corona for Mac OS X, you will receive an Apple disk image archive (`.dmg`) file. To install Corona, open the `.dmg` file and follow the on-screen instructions. Once installed, you will be presented with the **Corona Developer Login** screen with the option to sign in or register as follows:

Since we've already registered, enter the e-mail address and your password into the sign-in screen. Once you've signed in, you are ready to move on to the next steps of your mobile adventure!

Installing for Microsoft Windows

If you've downloaded Corona for Microsoft Windows, you will receive a Windows installer file. Open the installer file and follow the on screen steps to install Corona. Similar to the Mac OS X install process, you will have to sign in to the Corona Developer Login before you can start using the product.

Corona SDK has one extra requirement for the Microsoft Windows platform. In order to create device builds, you need to have Java 6 SDK installed. To install it, head over to `www.oracle.com/technetwork/java/javase/downloads/`.

> At this time, Corona SDK does not support JDK7. For any issues or more information on the installat ion of Corona SDK, head over to `http://docs.coronalabs.com/guide/`.

Using Corona SDK

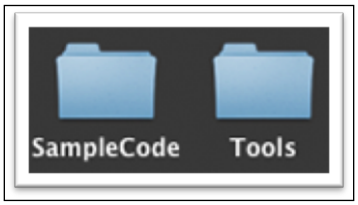
Now that you have the Corona SDK product installed on your computer, let's review the pieces that come with the product.

The **Corona Simulator** is the core product of Corona SDK. The simulator allows for the running of apps from your computer and allows you to preview how your app will look on a device:

Corona Terminal launches the simulator, but with an added feature. The terminal will display errors and warning messages that will help you debug your app:

Corona also offers a debugger that will run your app in the simulator. The debugger helps you troubleshoot by allowing you to specify events, create breakpoints, and much more:

When Corona is installed, you are provided with sample code sets and limited trials to third-party products. The sample code is especially helpful for individuals just starting with Corona SDK:



www.allitebooks.com

Once you've opened the Corona Simulator, you will be presented with the **Welcome to Corona** screen as follows:

The welcome screen contains links to **Sample Apps**, **APIs**, **Forums**, **Share Code**, and more tutorials. Besides the **Sample Apps**, one of the best features of the welcome screen is the ability to demo features. By clicking on **Demo**, Corona shows you how to incorporate some of their top features into your next app.

Summary

In this chapter, we were introduced to Corona SDK and learned what this product can offer us. We also learned more about the requirements of Corona, how to install Corona, and the tools that come with Corona SDK. In the next chapter, we will dive into Lua to give us a good starting foundation of the programming language.

2
Learning Lua

Do you have an idea for an app? Would you like to create a game for players around the world? Before you can run, you have to learn to walk, and your next step in your mobile adventure is to learn how to use Lua.

This chapter will provide you with a starting foundation for how to use Lua with Corona SDK. Here are the topics that we will be covering in this chapter:

- Using variables with Lua
- Using the terminal
- Expressions and operators
- Making decisions
- Loops, loops, and loops
- More flexibility with tables
- Using functions

Let's get started!

Using variables with Lua

With any mobile application, you will need a way to deal with information. Whether you need to store a player's level, name, or preferences, Lua provides you with a way to deal with information through variables.

Variables can store different types of information; the basic types are numbers, strings, and Boolean values:

- **Numbers** can be anything from a quantity to the current player level. You can perform basic math operations on number variables, such as addition, subtraction, and more.

  ```
  myAge = 28;
  ```

- A **string** is a set of characters, which can be anything including letters in the alphabet, numbers, and other characters. A string variable is easily identified by the double quotes surrounding the set of characters.

  ```
  myString = "My Mobile Adventure";
  ```

- The third type of information is a **Boolean value**. Boolean values can only be either true or false. A great example of a Boolean value is the mute button. If the player has decided to mute your app, the value for the mute variable would be true. Otherwise, the variable is set to false.

  ```
  muteButton = true;
  ```

There are some rules with variables that we need to cover:

- Lua is a dynamically typed language, and we do not need to specify the type of information that we are storing. Lua is smart enough to know when you want to store a string and when you want to store a Boolean value.
- All variables are case sensitive. Therefore, `myName` and `myname` are different variables.
- Lua has a list of reserved keywords that you cannot use when naming variables. Some of these reserved keywords are `if`, `then`, `else`, and `do`. For a full list of reserved keywords, visit `www.lua.org/manual`.

Using the terminal

Now that we know how to store variables, how do we know what data is inside a variable? Lua provides a `print()` command that gives you the ability to look inside variables (and tables, which we will get to later). Let's say we have the following variable in our app:

```
favoriteAnimal = "panda";
```

At some point, we will need to reference the variable, and an easy way to find out the contents of the variable is with the `print()` command.

```
print(favoriteAnimal)
```

If we use the `print()` command in our app, we will see the contents of the variable only in the Corona Terminal. Next, let's cover expressions and operators.

Expressions and operators

Just like any programming language, Lua supports basic arithmetic expressions and comparison operators. These are valuable tools in your app-making arsenal and will help you get the most out of Lua. Let's take a look at the expressions and operators that you can use in Lua.

Arithmetic expressions

Lua supports all the basic math functionalities: addition (+), subtraction (-), multiplication (*), and division (/). In addition to these basic functions, you also have the ability to use the modulus (%) and exponents (^) expressions. You can use the modulus expression to retrieve the remainder of a division, and the exponents expression will multiply a number by itself based on the value of the exponent.

Comparison operators

In addition to arithmetic expressions, you also have access to basic comparison operators. The comparison operators are as follows:

- Equality (==)
- Inequality (~=)
- Less than (<)
- Greater than (>)
- Less than or equal to (<=)
- Greater than or equal to (>=)

Making decisions

In life, we all have to make decisions. To say yes or to say no, to buy or not to buy, to shake or to stir, to... you get the point. Our apps also have to make decisions based on conditions, and Lua has provided you with a way to do so.

The if-then statement

Our first decision statement is the **if-then** statement. The if-then statement will perform an action based on the result of a condition. Every if-then statement follows a standard layout: test a condition, and if true, perform an action. Here's an example:

```
if(age < 29) then
print('You are not quite 30!');
end
```

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at `http://www.packtpub.com`. If you purchased this book elsewhere, you can visit `http://www.packtpub.com/support` and register to have the files e-mailed directly to you.

Let's break down our if-then statement so we understand what's happening:

1. We let our app know we are starting a decision process with the keyword `if`.
2. Next, we enclose a conditional statement with a pair of parentheses. Inside the parentheses, we are comparing two values to each other using the less than operator. When comparing a text value, we need to wrap the value with quotes; otherwise, don't use any quotes.
3. After the conditional, we use the keyword `then` to state that the action is to be performed only if the condition is true.
4. Finally, we wrap up the decision process with the keyword `end`.

In our sample if-then statement, we are printing the string `You are not quite 30!` to our Corona Terminal. If our variable age was 29 or higher, nothing would be printed to the terminal.

The if-then-else statement

What should you do when you have to make more than one decision? There's a decision statement for that too—the **if-then-else** statement.

Every if-then-else statement has a standard layout: test a condition, and if true, perform action 1; otherwise, perform action 2. Let's take a look at an example of an if-then-else statement:

```
if(player == "James Bond") then
print('Drink Shaken');
else
print('Drink Stirred');
end
```

And now let's look at a breakdown of the if-then-else statement so we can understand what's happening:

1. We start the decision process by using the `if` keyword.
2. Next, we enter a conditional statement. In our sample, we are asking our app to test the variable player to the string `James Bond`.
3. If the conditional statement is true, then the decision will perform the first action, which is to print `Drink Shaken` to our terminal. Otherwise, the decision will perform the second action (`else`), which says to print `Drink Stirred` to our terminal.

Both if-then and if-then-else statements are great for pieces of code that need to run conditionally.

Loops, loops, and loops

Take a moment and think back to the last game you played. Was it Pong? A racing game? Or was it a puzzle game? No matter what the genre was, all of these games had one thing in common: they all used **loops**.

In almost every app that you make from here on, your app is going to have a repetitive action. These actions can be anything from keeping track of the time remaining in a game to creating onscreen items. With loops, you can avoid copying and pasting dozens of lines of code and save yourself a lot of time.

The for loop

The first loop that we are going to cover is the **for loop**. For loops are great for a predefined number of repetitions. If you know that you need to make a player take three steps in a direction, a for loop is perfect for that situation. All for loops stick to a standard layout: initialize, test, and update. Let's take a look at a sample:

```
for i=1,3 do
  print(i);
end
```

Now let's look at a breakdown of the for loop so we know what's happening:

1. We start the loop by using the `for` keyword to let the app know we want to start a loop.
2. Next, we initialize the variable `i` and we assign it the value `1`.
3. After the initialization, we put in our test condition. In this case, we are telling our app that we want to loop until `i` reaches `3`.
4. What we don't see in our for loop is the update. By default, every time our app goes through the loop, the variable `i` will be increased by one.

If we ran our sample code in an app, we would see the following in our terminal.

```
1
2
3
```

We see three numbers here because we told the for loop to run until the variable `i` equaled `3`. Once the variable `i` reached `3`, the for loop stopped running. Let's take another look at the for loop, but this time in a real-world example:

```
playerStepsTaken = 0;
distanceTraveled = 0;
for i = 1,10 do
  playerStepsTaken = playerStepsTaken + 1;
  distanceTraveled = distanceTraveled + 5;
end
```

This code may look a little foreign at first; however, it's not as hard as it seems. We start by creating two variables—`playerStepsTaken` and `distanceTraveled`—and assign them the value `0`. Next, we know that we are using a for loop to accomplish a task 10 times, and in our example, we are increasing the `playerStepsTaken` variable's value by 1 and the `distanceTraveled` variable's by 5.

The power behind for loops is the ability to perform several repetitive actions with fewer lines of code. Instead of writing 20 lines of code, we are able to perform our action in four lines (six including initializing the variables). Imagine how useful a for loop is when you have to keep track of a player over 100 steps, or even 1,000!

The while loop

The next loop that we will cover is the **while loop**. The while loop is better suited for an unknown number of repetitions, such as telling a player to take steps until they reach their goal. All while loops follow a standard layout: testing a condition and performing an action if the condition is true. Let's take a look at a sample:

```
timer = 0
while timer < 10 do
print(timer);
timer = timer + 1;
end
```

Now let's look at a breakdown of the while loop so we know what's happening:

1. Before we start the loop, we need a variable to test in our condition. In this sample, we created a variable named `timer` and assigned it the value `0`.
2. Next, we start the loop by using the `while` keyword to let our app know we are entering a while loop. This line also consists of the test variable and the test condition. In our sample loop, we are telling our loop to stop when the variable timer reaches the number `10`.
3. In the next line, we print the value of `timer` to the terminal just so we know what's happening to the variable.
4. After the print, we increase the value of the `variabletimer` variable by 1. We increase this so we can provide a way for our loop to exit. Otherwise, our loop would run forever!
5. Finally, we use the keyword `end` to tell our app we are done writing our loop action.

In the Corona Terminal, we would see the numbers 1 through 10.

More flexibility with tables

So far, we've used one variable to store one piece of data at a time. This works great when we are dealing with a small amount of information. But what happens when you have to deal with hundreds of variables? When you start to deal with a large number of variables, your code can start to get sloppy.

With Lua, you have access to **tables**. Tables are a better way to manage several pieces of information in one location. You can think of a table as a filing cabinet with different drawers. Each drawer within the cabinet has different files with different information. Also, each drawer has a way to identify itself with a piece of information, called the index. Let's take a look at how to create a table with Lua:

```
myTable = {1, "apple", 3}
```

Pretty simple, right? We created a table by declaring the table name followed by closed curly brackets. Inside the curly brackets, we add data to the table just like we would with a variable, except separated by commas. In our simple table, the number `1` is stored in slot 1 (drawer 1), the string `apple` is stored in slot 2 (drawer 2), and the number `3` is stored in slot 3 (drawer 3).

The location of the information is referred to as the index, and we can add more data to the table by defining the index of any new data. In this example, we add the string `My Mobile Adventure` to the fourth index of our table:

```
myTable[4] = "My Mobile Adventure"
```

If we want to access our data in the table, we do so by typing the name of the table followed by the index wrapped in square brackets:

```
print(myTable[4])
```

Using a number as the index is just one way to add data to a table. We can also add data to a table using a name as the index. Using a name instead of a number as the index (the storage location of the data) makes data easier to read later on. Instead of using a number to represent a storage location, we can use a name. Let's take a look at an example:

```
namedTable = {
name = "Dan",
sdk = "Corona"
}
```

By using a named index instead of a numerical index, we make our table easier to read, not only for ourselves, but for anyone else working on the app. If we want to access the data in our named table, we use the name of the index instead of the number:

```
print(namedTable["name"])
print(namedTable["sdk"])
```

So far, we haven't seen much difference between storing data in a variable and storing it in a table. So, let's take a look at how we can get the most out of a table.

Consider this: if you need to create 15 variables, how would you go about the task? Would you use 15 lines to create variables? Or would you use a combination of what you've learned to work smarter and not harder? Here's one way to create 15 variables quickly and easily:

```
myTable = {}
for i = 1,15 do
myTable[i] = i
end
```

In this example, we created a table and established a for loop to execute 15 times. During each execution, we store the current loop value into the table.

Here's another example of the power of tables and loops. Let's say you have a business app that helps people convert 20 numbers from feet to meters. You could write several lines of code to convert every number, or you could write a simple for loop with four lines of code:

```
for i=1,20 do
metersTable[i] = feetTable[i] * 0.3048
print(metersTable[i])
end
```

With the assumption that you have already created the variables `metersTable` and `feetTable`, this for loop will convert the contents of `feetTable` to meters and store it in `metersTable`.

Using functions

One of the most powerful features of any programming language is the ability to use functions. Functions are blocks of code that perform a specific task and can be used multiple times. Functions can be useful time savers because you can use the same block of code throughout your app, and if you need to make changes, you have to make it at only one place. Let's take a look at how they work.

A function is defined by starting with the keyword `function` followed by the name of the function. Any lines of code between the name of the function and the keyword `end` will run when the function is called. To call a function, we simply use the name of the function followed by open and closing parenthesis.

```
function gameTimer()
print(timer)
end
gameTimer()
```


www.allitebooks.com

In the example here, we created a function called `gameTimer`; it prints the value of the variable `timer` to the Corona Terminal. Then, we call the function by writing `gameTimer()`.

Another great feature of functions is the ability to pass information to the program through arguments. By making your functions accept outside information, you are creating flexible functions that can be used to fit more scenarios. Let's take a look at a function with arguments:

```
function addNumbers(one, two)
print(one + two)
end
addNumbers(1,2)
```

In this example, we created a function called `addNumbers`, but this function accepts two arguments called `one` and `two`. These arguments can then be used as variables within our function to perform an action. In this case, we are adding two numbers together. To make use of the arguments, we pass the numbers that we want to use in our function by using them in the call line `addNumbers(1,2)`. If we want to add different numbers, we just replace the numbers `1` and `2` with something else, for example, `addNumbers(10,18)`.

Functions are a very powerful feature, and while we haven't covered everything functions can do, we have a great foundation to start from.

Summary

In this chapter, we covered how to use variables, the print command, expressions, operators, decisions, loops, tables, and functions. We also saw examples of each topic to better understand how to use what we learned with Lua.

Since this chapter is only intended to serve as a starting point in your mobile adventure, we only skimmed the surface of what Lua is capable of. If you would like to learn more about Lua, I strongly suggest visiting `www.lua.org`, or heading over to `www.coronalabs.com/resources` for additional learning. In the next chapter, we will discuss the steps involved in developing our very first app.

3
Creating Your First App

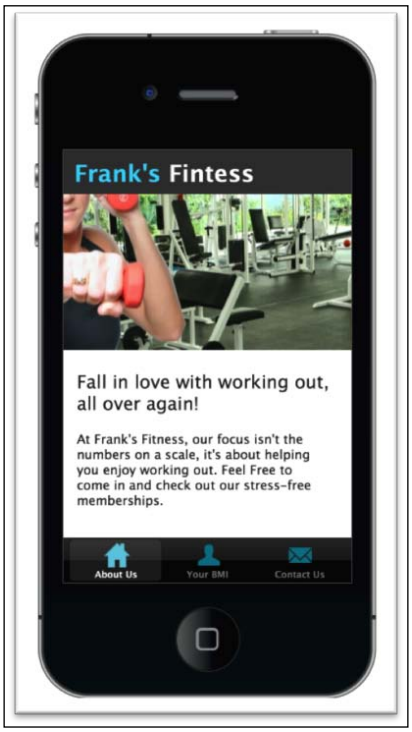
Too many programming books start off with the rigors of learning the background of the programming language and teaching you what seems like every piece of functionality even before you make your first "Hello World" program. Well, this book isn't one of them.

Inside this chapter, you will be diving right into making your very first app within minutes. Here's what we will be covering:

- Using a text editor
- Creating a new project
- An introduction to Storyboard
- Using widgets
- Using display objects
- Creating the views for your project
- Configuring your project

For our first app, *Frank's Fitness*, you have access to a starter kit that includes the graphics used throughout this app. You can get the starter kit along with the code files available with this book. Let's get started!

Welcoming you to Frank's Fitness:



The first app in our mobile adventure will be a business-style application for a local gym—Frank's Fitness. Inside this app, people will be able to learn more about Frank's Fitness, calculate their body mass index (BMI), and easily contact customer service.

We will also be discovering how to use Corona's Storyboard feature, widgets, and touch events, as well as how to send e-mail. For now, let's discuss how to use a text editor.

Using a text editor

We'll be spending some time editing our code, and for that we'll need a text editor. A text editor is a program that allows you to edit files, but there are certain text editors that you can use for programming. These programming-specific text editors will provide auto-completion, syntax highlighting, and a whole lot more. Here's a list of great text editors that are free or have paid licenses:

Notepad++

This is a free text editor that supports Lua. Currently, Notepad++ is only available on Windows, but if you are on a Mac, **Text Wrangler** is a free editor that's just as capable. Learn more at `http://notepad-plus-plus.org/`.

Sublime Text

This is a little more sophisticated than Notepad++, and is available with either a trial or paid-for licensing. Sublime Text has great shortcuts that can save you a lot of time with debugging and repetitive tasks. Sublime Text is also cross-platform, so you can use the editor on Windows or Mac. Learn more at `http://www.sublimetext.com/`.

Corona-specific editors

There are a few text editors that are specific to Corona SDK, and they provide more advanced functions that are out of the scope of this book. However, if you would like an editor that's made specifically for Corona SDK, here are some:

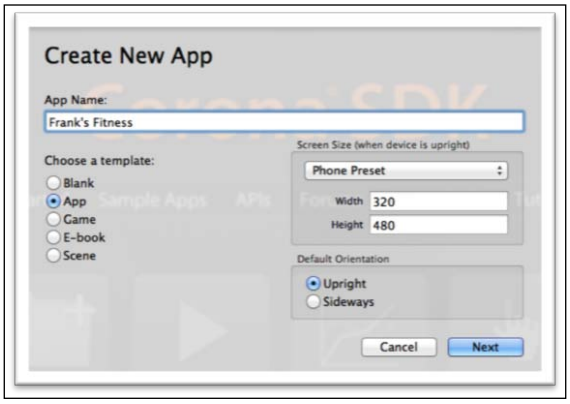
- **Corona Cider**: `http://www.mydevelopersgames.com/CIDER/`
- **Corona Complete**: `http://coronacomplete.com/`
- **Outlaw IDE**: `http://outlawgametools.com`

Creating a new project

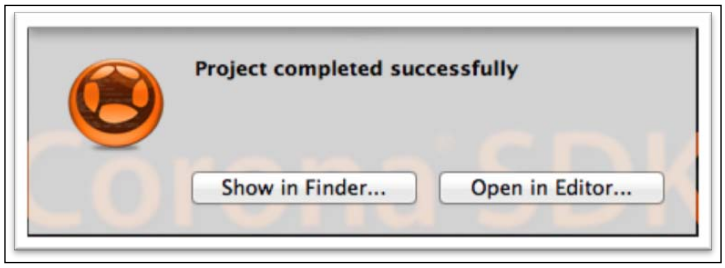
Now that we've learned about the app and some text editors, let's set up our project. Corona SDK has made it very easy for developers to start a new project by providing a button labeled **New Project** on the welcome screen.

To start the new project, click on the folder icon labeled **New Project**. Once you click on it, you will be presented with a screen that asks for the app name, template type, screen size, and default orientation. Let's fill in the following information:

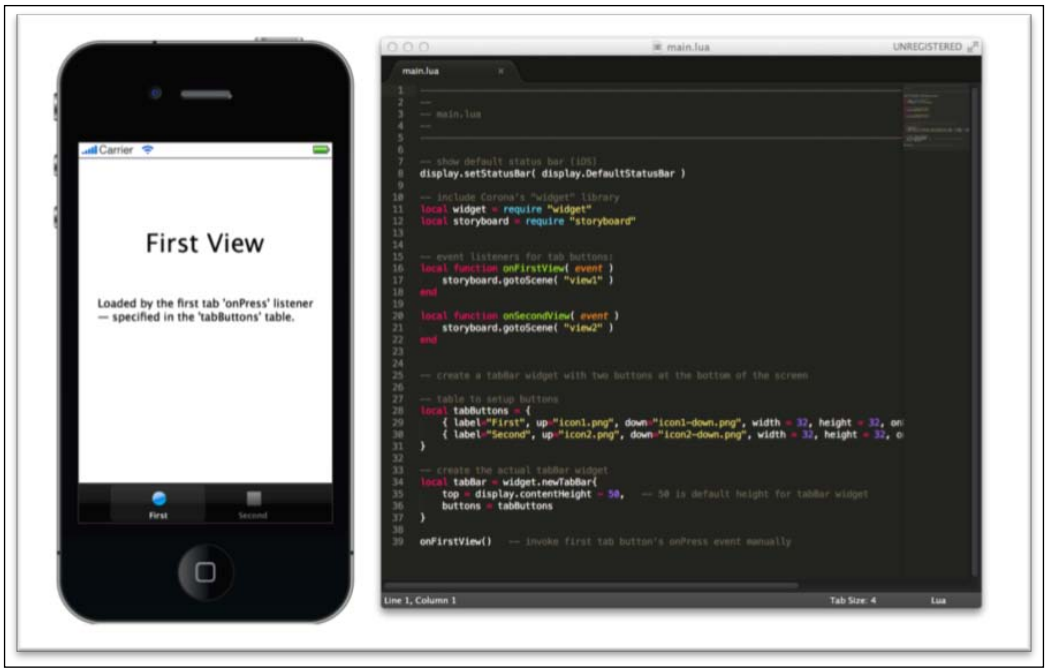
- **App Name**: **Frank's Fitness**
- **Choose a template**: **App** (or **Tab Bar Application** for Windows users)
- **Screen Size**: **Phone Preset**
- **Default Orientation**: **Upright**



In the next screen, you will be asked to choose the location of your project. Select a convenient location for your project and click on **Create**. You will then have the options **View the Folder** or **Open the Project in Editor** to choose from. For Windows users: you can specify the location of your project when it's created.



Choose the option **Open in Editor…** and this will bring up the project in the Corona Simulator and the project code in your favorite text editor. If you are on Windows, you will see a new window with a list of project files that make up your app.



Congratulations! You've created your very first app with Corona SDK! To start with, our app displays two tabs titled `First` and `Second`. When you touch a tab on the screen, you will move to the respective tab's view. Although our app doesn't do much yet, we will be adding functionality throughout the chapter.

Before we dive into the code of the app, let's take a look at the files that make up the project. Open up the location of the project, and inside the project folder you will see 13 files. Let's find out what each one does:

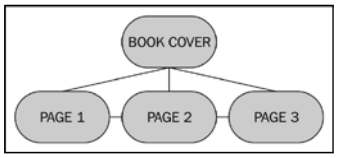
- `build.settings`: This file handles the device orientation, auto-rotation behavior, and other build-time settings that you may need to set. For our first app, we will not need to edit this file, since Corona has already set it up for us.
- `config.lua`: If `build.settings` is used for build-time settings, the `config.lua` file is used for run-time settings. Inside this file, you will be able to set the width and height of your app, the type of auto-scaling, and the frames per second. You can also take advantage of Corona's dynamic image resolution. This feature allows you to swap out higher-resolution versions of your images when using your app on a higher-resolution device.

- `icon1.png` and `icon2.png`: As part of the sample app, Corona gives you eight images to use. These images are used for the first and second tab at the bottom of the app. Also, Corona has provided a high-resolution version of each image. Each high-resolution image is marked with `@2x` at the end of the image's name. We will be discussing how to use higher-resolution images later.
- `main.lua`: This file is going to be the starting point of every app you make with Corona SDK. When you open up an existing project or create a new one, you'll always start with the `main.lua` file. This file is a great place to include libraries or other sets of code to bring more interaction to your app.
- `view1.lua` and `view2.lua`: These files represent the `First View` and the `Second View` that you see within the app. In the final version of Frank's Fitness, we will modify the first two views and create a `Third View` later on.

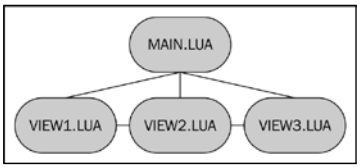
An introduction to Storyboard

Now that we know how our app is structured, we need to understand how Corona switches between the different views. Corona uses an API called Storyboard to manage all views, or scenes, within your app.

The easiest way to visualize Storyboard is by visualizing your app as a book. Every book has a cover and pages that are filled with content. When you start reading the book, you start with the cover, and then you can choose to begin at page 1 or flip to a page in the middle. At any point in the book, you can flip to any other page to re-read passages. This is exactly how Storyboard works.



In the previous flowchart, the book starts with the book cover and leads to pages 1, 2, and 3. If you start reading at page 1, you can go to page 2 or 3 at any time and vice versa. With Storyboard, the `main.lua` file is the book cover, or the starting point, and every scene is considered a page within the book. So far, we have titled each scene as `view1.lua`, `view2.lua`, and `view3.lua`.



Storyboard is a great tool provided by Corona that makes managing your app easier. This tool provides a structure to your app by making the `main.lua` file the starting point and each view file another page of content within your app.

> Only the `main.lua` file has to be named `main.lua`. All other view files can be named as you see fit, for example, `MainMenu.lua` or `bmi.lua`, or anything you wish.

Now that we understand how the Storyboard tool operates, let's look at how to implement the Storyboard feature within our app. In order to start using Storyboard, we have to first "require" the Storyboard library and create a variable that uses the library. We accomplish this using the following lines of code:

```
local storyboard = require "storyboard"
local scene = storyboard.newScene()
```

These lines are required in any view that uses the Storyboard feature. Every scene has four functions: creating a scene, entering a scene, exiting a scene, and destroying a scene. Corona then uses an event listener to call each function based on what is happening within your app. An event is simply an action that happens within your app; in this case, it's when you go to a new scene or leave a scene. We'll have a look at the code for this next.

The createScene function

The `createScene` function is called when moving to a new scene for the first time. A lot of the setup work is done within the `createScene` function, such as creating objects that will go on the screen.

```
function scene:createScene( event )
end
scene:addEventListener( "createScene", scene )
```

The enterScene function

The `enterScene` function is called after a scene has moved on to the screen. This function is the perfect place for any app or game logic that makes your scene operate. For example, if you have to calculate the BMI, the logic for the calculation would go in `enterScene`.

```
function scene:enterScene( event )
end
scene:addEventListener( "enterScene", scene )
```

The exitScene function

The `exitScene` function is the janitor function, and is called when you are leaving a scene. In this function, you'll be removing any widgets, timers, and other Corona-specific features.

```
function scene:exitScene( event )
end
scene:addEventListener( "exitScene", scene )
```

The destroyScene function

The `destroyScene` function is called when you have to destroy a scene in low memory situations. By destroying a scene, you are removing the scene from the mobile device's memory.

```
function scene:destroyScene( event )
end
scene:addEventListener( "destroyScene", scene )
```

Switching scenes

Switching between scenes in our app is easy. To switch scenes, use the keyword `storyboard.gotoScene()` with the name of the scene in double quotes.

```
storyboard.gotoScene( "scene1" )
```

Destroying scenes

Every time a scene is created, that scene stays in the device's memory for the duration of the app. If you have too many scenes in the device's memory, the performance of your app can be adversely affected. You can avoid this by destroying previous scenes that are no longer required.

```
storyboard.removeScene( "scene1" )
```

Or, you can remove all for this next scenes within you app by using the following line:

```
storyboard.removeAll()
```

Although you can manage the memory on your own, Corona provides a feature called **automatic scene purging** that will destroy the least recently-used scenes to help with memory management.

If you would like to learn the ins and outs of Storyboard, I suggest visiting `http://docs.coronalabs.com/api/library/storyboard/index.html`. Corona Labs provides a scene template along with all the in-depth details regarding scene management, scene transitions, and more.

Using widgets

Another feature of Corona that we need to understand before diving into our code is the widget. Corona provides common interface widgets such as buttons, sliders, a picker wheel, and a tab bar.

In our Frank's Fitness app, we are going to use the tab bar to provide an intuitive navigation for our users. The tab bar creates a customizable bar with tab buttons at the bottom of the screen.

With our starter app, we have two buttons titled **First** and **Second**. To create a tab bar, we have to let our app know we want to start using widgets by requiring the widget library.

```
local widget = require "widget"
```

> When we created our app using Corona's **New Project** feature, most of this code had already been created for us.

Now that our app knows we want to use the widget library, we create a table that holds our tab information.

```
local tabButtons = {
{
  label="First",
  defaultFile="icon1.png",
  overFile="icon1-down.png",
  width = 32,
  height = 32,
  onPress=onFirstView,
  selected=true
},
```

```
{
  label="Second",
  defaultFile="icon2.png",
  overFile="icon2-down.png",
  width = 32,
  height = 32,
  onPress=onSecondView
},
}
```

Here's a breakdown of what we are using in our table:

- `label` (optional): This is the display name below the icon on our app.
- `defaultFile` and `overFile` (required): These define the images that we want to use for when the tab is active or inactive.
- `width` and `height` (optional): This defines the width and height of our tabs in pixels.
- `onPress` (optional): This function is called when the tab is touched by the user.
- `selected` (optional): This determines which tab is active when the tab bar is created.

Next, we create the actual tab bar widget by creating a new variable and assigning it to `widget.newTabBar`.

```
local tabBar = widget.newTabBar{
top = display.contentHeight - 50,
buttons = tabButtons
}
```

Here's a breakdown of how we are using `widget.newTabBar`:

- `top` (required): This defines the location of the tab bar on our app. By using `display.contentHeight - 50`, we are telling Corona to place the tab bar 50 pixels above the bottom of the screen.
- `buttons` (required): We use the table `tabButtons` that was created for us with `buttons`. This lets `newTabBar` know how we want our buttons to look.

Once you create the table and variable for the widget, you will see the tab bar on your app. To learn about the other widgets that Corona offers, visit `http://docs.coronalabs.com/api/library/widget/index.html`. On the documents page, you will learn about Corona's widgets for buttons, a picker wheel, a spinner, and more.

Starting with main.lua

Now that we've learned about the structure of our app as well as Storyboard, we can finally start looking at the code inside our app! If you've been following the book, both the simulator and your text editor should already be open; if not, open `main.lua` with the Corona Simulator and open it again with your favorite text editor. Inside your text editor, you will see 39 lines of code that make up the `main.lua` file.

```
1  ------------------------------------------------------------------------------
2  --
3  -- main.lua
4  --
5  ------------------------------------------------------------------------------
6
7  -- show default status bar (iOS)
8  display.setStatusBar( display.DefaultStatusBar )
9
10 -- include Corona's "widget" library
11 local widget = require "widget"
12 local storyboard = require "storyboard"
13
14
15 -- event listeners for tab buttons:
16 local function onFirstView( event )
17     storyboard.gotoScene( "view1" )
18 end
19
20 local function onSecondView( event )
21     storyboard.gotoScene( "view2" )
22 end
23
24
25 -- create a tabBar widget with two buttons at the bottom of the screen
26
27 -- table to setup buttons
28 local tabButtons = {
29     { label="First", up="icon1.png", down="icon1-down.png", width = 32, height = 32, onPress=onFirstView, selected=true },
30     { label="Second", up="icon2.png", down="icon2-down.png", width = 32, height = 32, onPress=onSecondView },
31 }
32
33 -- create the actual tabBar widget
34 local tabBar = widget.newTabBar{
35     top = display.contentHeight - 50,   -- 50 is default height for tabBar widget
36     buttons = tabButtons
37 }
38
39 onFirstView()   -- invoke first tab button's onPress event manually
```

Before we start editing `main.lua`, let's take a look at the pre-existing code to understand what's going on:

- Lines 1-5: `main.lua` starts out with five commented lines. Commented lines are marked with two dashes and are used only for keeping notes within your app.
- Line 8: `display.setStatusBar( display.DefaultStatusBar )` is only applied in the iOS system; this line of code is saying to show the status bar at the top of the screen.
- Lines 11-12: Here, our app is telling Corona to start using the widget and Storyboard library.

- Lines 16-22: Moving along `main.lua`, you'll come across two functions. These functions will be called when the user touches a tab icon.
- Lines 28-31: This set of code creates a table that saves how we want our tabs to look.
- Lines 34-37: Now, our code is adding the tab bar to the app.
- Line 39: Finally, our code is calling the function `onFirstView()`. This function will make `view1` our default view by telling Storyboard to move to `view1.lua`.

Now that we know how our app is working, we are going to start adding some functionalities. The first change we want to make is on line 8. Instead of showing the status bar, let's hide it when our app is open.

```
display.setStatusBar( display.HiddenStatusBar )
```

After you change this line, you should no longer see the status bar.

> With Corona, you will need to relaunch the simulator before you see any change that you have made in your code. You can relaunch the simulator with the keyboard shortcut *Ctrl* + *R* for Windows, or *Cmd* + *R* for Mac.

Next, we are going to add a background to our app by creating display objects. Anything that is drawn on the screen, such as rectangles, circles, or images, is to be considered as a **display object**. When display objects are added to the screen, they are applied in the same way a painting is created. When you paint the first layer on the canvas, the paint layer will show up on the top. If you add a second layer, the second layer will lay on top of the first, and so on. Just like painting on a canvas, if you apply a second display object over the first display object, only the second display object will be seen.

For our background, we are going to create three rectangle display objects and add them to the screen. Once they are on the screen, we will change the color of the rectangles and make sure the rectangles are at the back of the screen. Here's how we create a rectangle display object:

```
myVariable = display.newRect(top, left, width, height)
```

In this code, we started with a variable and assigned it to `display.newRect()`. This function creates a new rectangle and accepts four arguments. The first two arguments are for the top-left coordinate in X-Y format, and the last two arguments define the width and height of the square. Next, we can use a built-in function for display objects to change the color:

```
myVariable:setFillColor(red, green, blue)
```

By default, the fill color of display objects is set to white, or (255, 255, 255). We want our rectangle a dark grey, so we set the arguments of `setFillColor` to (49, 49, 49). Finally, we will push the rectangle to the back of the screen by using the `toBack()` function on our rectangle:

```
myVariable:toBack()
```

Let's start by adding the first rectangle after the local requires of widget and Storyboard:

```
rect_upperBackground = display.newRect(0,0,320,175)
rect_upperBackground:setFillColor(49,49,49)
rect_upperBackground:toBack()
```

In this code, we are adding a rectangle to the screen, changing the fill color, and sending it to the back of the screen. Now that we've added one rectangle, let's add two more lines to the screen after `rect_upperBackground:toBack()` to make our app look better:

```
rect_middleBackground = display.newRect(0,175,320,2)
rect_middleBackground:setFillColor(0,0,0)
rect_middleBackground:toBack()
rect_background = display.newRect(0,0,320,480)
rect_background:toBack()
```

[You may not see any big changes yet, but they're coming!]

After we've added a background to our app, our next task is to make some changes to the event listeners for our tab buttons. Currently, we have two event listeners that will change the view when a button is touched. We need to modify the functions to not change when the user is already on the current view. So, if the user is on `view1` and they touch the button for `view1`, we will not reload the view because they are already there. Also, we will need to add another event listener for our third button. Let's add the following lines of code to `main.lua`:

```
-- event listeners for tab buttons:
local function onFirstView( event )
  currentScene = storyboard.getCurrentSceneName()
  storyboard.gotoScene( "view1" )
end

local function onSecondView( event )
  currentScene = storyboard.getCurrentSceneName()
  storyboard.gotoScene( "view2" )
end
```

```
local function onThirdView( event )
  currentScene = storyboard.getCurrentSceneName()
  storyboard.gotoScene( "view3" )
end
```

In this section, we used the `getCurrentSceneName()` function from Storyboard. This function is simply getting the name of the current scene (`view1`, `view2`, or `view3`) and assigning it to the string `currentScene`. This is an important piece of functionality because it allows our users to navigate through the Frank's Fitness app.

Finally, we are going to modify the table `tabButtons`. We will be changing the display name and images for the tab. The images that we used are included in the starter kit.

```
local tabButtons = {
  {
label="About Us",
defaultFile="tab-about.png", overFile="tab-about-down.png",
  width = 32, height = 32,
  onPress=onFirstView,
selected=true
},
  {
label="Your BMI",
defaultFile="tab-bmi.png", overFile="tab-bmi-down.png",
  width = 32, height = 32,
  onPress=onSecondView
},
  {
label="Contact Us",
defaultFile="tab-contact.png", overFile="tab-contact-down.png",
  width = 32, height = 32,
onPress=onThirdView
}
}
```

If you've been following along, your app should look like the following screenshot. We've created a simple background and added three buttons to our tab with custom images. In the following section, we will be adding functionalities to each view.

view1.lua – about us

The first view will be all about Frank's Fitness, and it will be the default view that users are sent to when they open the app. Let's start editing the code for `view1.lua` by opening the file with your favorite text editor.

When you open the file, you will see that `view1.lua` is using Corona's Storyboard feature and has structured the file using the `createScene`, `enterScene`, `exitScene`, and `destroyScene` functions. We will be doing most of our work inside the `createScene` function. Our first change is to remove the following lines from `view1.lua`:

```
-- create a white background to fill screen
local bg = display.newRect( 0, 0,
display.contentWidth, display.contentHeight )
bg:setFillColor( 255 )  -- white
```

These lines are used to create a background for our view. Since we have already created one in `main.lua`, we no longer need it. Next, we will add four variables in the place of the background variable. These variables will hold the content for `view1.lua`. By keeping the content separated, we are making the content easier to change later on.

```
namePart1 = "Frank's"

namePart2 = "Fitness"

motto = "Fall in love with working out, all over again!"

aboutFranksFitness = "At Frank's Fitness, our focus isn't
  the numbers on a scale; it's about helping you enjoy
  working out. Feel free to come in and check out our
  stress-free memberships."
```

Now that we've created our variables, we will use them by adding text display objects to the screen. In our current `view1.lua` file, we already have two `display.newRetinaText` objects. These display objects allow us to add text to the screen using the following format:

```
myVariable = display.newRetinaText( "my text", left, top,
  width, height, font, size)
```

This display object is similar to the rectangle display object, except that it allows you to set the top-left coordinates, the width, and the height. It also accepts three additional arguments used for the text, the type of font, and the size of the font. We are going to remove the display objects that are currently in `view1.lua` and replace them with the following:

```
txt_name1 = display.newText( namePart1, 0,
  0, 176, 100, native.systemFontBold, 26 )
  txt_name1:setTextColor( 81, 212, 252 )
  txt_name1.x = 100
  txt_name1.y = 60
  group:insert( txt_name1 )

  txt_name2 = display.newText( namePart2, 0,
  0, 176, 100, native.systemFontBold, 26 )
  txt_name2:setTextColor( 255, 255, 255 )
  txt_name2.x = 205
  txt_name2.y = 60
  group:insert( txt_name2 )

  txt_logo = display.newText( motto, 0, 0,
  300, 100, native.systemFont, 20 )
```

```
txt_logo:setTextColor( 22, 22, 22 )
txt_logo.x = 165
txt_logo.y = 295
group:insert( txt_logo )

txt_about = display.newText( aboutFranksFitness,
0, 0, 280, 136, native.systemFont, 14 )
txt_about:setTextColor( 22, 22, 22 )
txt_about.x = 155
txt_about.y = 380
group:insert(txt_about)
```

With our text display objects, we introduced a new line of code: `group:insert(txt_about)`. For more information about groups, Corona has a great introduction at `http://www.coronalabs.com/blog/2012/02/28/corona-display-groups-101/`. For every new display object that we create, we are adding them to the `group` variable. We add them to the `group` variable to let our app know that the respective display object belongs to the current scene. When we switch scenes, all display objects that belong to the `group` variable will be handled accordingly (that is, removed to free up memory).

To make our app more appealing, we are going to add an image of a gym to the home page. We can add images to the screen by using the image display object. Here's an example:

```
myVariable = display.newImageRect( filename,
width, height )
```

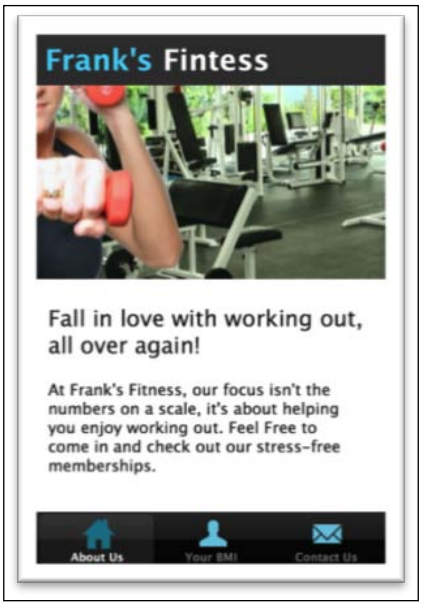
Using the image display object, we will add our image to our app:

```
img_gym= display.newImageRect(
"assets/scene1/gym.jpg", 400, 173)
  img_gym.x = display.contentWidth/2
  img_gym.y = 135
  group:insert(img_gym)
```

Finally, we need to remove the following four lines from `view1.lua`. We need to remove these four lines because we are no longer using these display objects in our app.

```
-- all objects must be added to group (e.g. self.view)
  group:insert( bg )
  group:insert( title )
  group:insert( summary )
```

Once you relaunch your app, the app for Frank's Fitness should look like the following screenshot. We used the rectangle display object to create a background, and we've added some images and more information about the gym to `view1.lua`. Next, we will be creating a BMI calculator in `view2.lua`.



view2.lua – your BMI

The second view will contain a body mass index calculator that will allow users to input their height and weight to calculate their BMI. Let's get started by opening `view2.lua` in your favorite text editor.

Once you have `view2.lua` open, you can see that it was built the same way as `view1.lua`. `view2.lua` uses the Storyboard feature, and most of the code is in the `createScene()` function. Instead of modifying the code with the `createScene()` function, let's remove everything from inside the `createScene()` function. Our function should look like this:

```
function scene:createScene( event )
  local group = self.view
end
```

Now that we've cleared our function, let's add eight variables that will hold the content for `view2.lua`. Add the following variables after local group:

```
bmiDefinition = "BMI, or body mass index, is a way to estimate your
body fat percentage based on your weight and height. To estimate your
BMI, enter your height and weight."
  heightDescription = "Your Height"
  weightDescription = "Your Weight"
  bmiDescription = "Your BMI"
startingBMI = "24.4"

  bmiUnderweight = "Underweight Below 18.5"
  bmiNormal = "Normal 18.5 - 25"
  bmiOverweight = "Overweight 25 - 30"
  bmiObese = "Obese 31+"
```

After we've added our content variables to our app, let's add the text display objects to `view2.lua` inside `createScene()`. We will be adding nine text display objects to the screen.

```
txt_bmi = display.newText( bmiDefinition, 0, 0, 160, 180, native.
systemFont, 13 )
  txt_bmi:setTextColor( 240, 240, 240 )
  txt_bmi.x = 232
  txt_bmi.y = 120
  group:insert( txt_bmi )

txt_height = display.newText( heightDescription, 0, 0, 100, 90,
native.systemFontBold, 16 )
  txt_height:setTextColor( 22, 22, 22 )
  txt_height.x = 85
  txt_height.y = 245
  group:insert( txt_height )

txt_weight = display.newText( weightDescription, 0, 0, 106, 90,
native.systemFontBold, 16 )
  txt_weight:setTextColor( 22, 22, 22 )
  txt_weight.x = txt_height.x
  txt_weight.y = txt_height.y + 70
  group:insert( txt_weight )
```

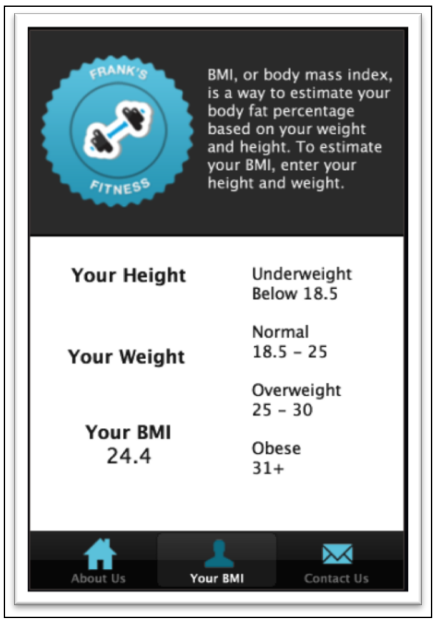

www.allitebooks.com

```
txt_bmi = display.newText( bmiDescription, 0, 0, 106, 90, native.
systemFontBold, 16 )
  txt_bmi:setTextColor( 22, 22, 22 )
  txt_bmi.x = txt_height.x + 15
  txt_bmi.y = 380
  group:insert( txt_bmi )

txt_calculateBMI = display.newText( startingBMI, 0, 0, native.
systemFont, 18 )
  txt_calculateBMI:setTextColor( 22, 22, 22 )
  txt_calculateBMI.x = txt_height.x
  txt_calculateBMI.y = 365
  group:insert( txt_calculateBMI )

txt_bmiUnderweight = display.newText( bmiUnderweight, 0, 0, 100, 90,
native.systemFont, 14 )
  txt_bmiUnderweight:setTextColor( 22, 22, 22 )
  txt_bmiUnderweight.x = 240
  txt_bmiUnderweight.y = 245
  group:insert( txt_bmiUnderweight )

txt_bmiNormal = display.newText( bmiNormal, 0, 0, 80, 90, native.
systemFont, 14 )
  txt_bmiNormal:setTextColor( 22, 22, 22 )
  txt_bmiNormal.x = txt_bmiUnderweight.x - 10
  txt_bmiNormal.y = txt_bmiUnderweight.y + 50
  group:insert( txt_bmiNormal )

txt_bmiOverweight = display.newText( bmiOverweight, 0, 0, 80, 90,
native.systemFont, 14 )
  txt_bmiOverweight:setTextColor( 22, 22, 22 )
  txt_bmiOverweight.x = txt_bmiUnderweight.x - 10
  txt_bmiOverweight.y = txt_bmiNormal.y + 50
  group:insert( txt_bmiOverweight )

txt_bmiObese = display.newText( bmiObese, 0, 0, 70, 90, native.
systemFont, 14 )
  txt_bmiObese:setTextColor( 22, 22, 22 )
  txt_bmiObese.x = txt_bmiUnderweight.x - 15
  txt_bmiObese.y = txt_bmiOverweight.y + 50
  group:insert( txt_bmiObese )
```

Now that we've added our content to the screen, let's add the logo for Frank's Fitness. The logo should be included in the starter kit and should be placed in the root directory of your code.

```
logo = display.newImageRect( "logo.png", 135, 135)
  logo.x = 75
  logo.y = 86
  group:insert( logo )
```

After saving and relaunching the simulator, your app should look like the following screenshot. We've added content and a logo to the screen. Next, we'll be adding the functionality for the body mass index calculator by allowing the user to enter their height and weight.



Everything we've done so far in `view2.lua` has happened in the `createScene()` function. Now that we are going to add some logic to the app, we will make certain additions to the `enterScene()` function. Inside the `enterScene()` function, we will add the user input fields, calculate the user's BMI, and add some event listeners.

Our first additions to the `enterScene()` function are two text fields that accept input from the users. In this case, we will be accepting numeric values only. We can add a text field to our app by using the following line of code:

```
userInput = native.newTextField(left, top, width, height)
```

Native user interface objects are treated just like display objects in Corona, but they act a little differently. These objects will always appear at the top of our canvas, which means that they will always appear above other display objects. Additionally, native user interface objects do not work with groups, and therefore they have to be explicitly removed. We will handle their removal in our `exitScene()` function.

With native user interface objects, we are able to specify additional properties to make them work the way we want to. Let's look at some of the properties that we can change:

- `userInput.inputType = "numeric"`: By specifying the input type, we can exercise some control over what the user is able to enter. By setting the type to `numeric`, we are telling our app to pull up a numbers-only keyboard. You can also set the input type to `default`, `phone`, `url`, and `email`.
- `userInput.size = "18"`: The size property controls the font size of the text inside the input box.
- `userInput.text = "70"`: The text property sets the default content for the input box.
- `userInput.align = "center"`: The align property sets the alignment of the text inside the input box. You can set this to left, center, or right.

For further information about the properties that you can set with text fields, visit `http://docs.coronalabs.com/api/library/native/newTextField.html`.

Inside our `enterScene()` function, let's add two native user interface objects to accept a height and weight:

```
input_height = native.newTextField(0,0,100,26)
input_height.x = 85
input_height.y = 235
input_height.inputType = "numeric"
input_height.size = 18
input_height.text = "70"
input_height.align = "center"

input_weight = native.newTextField(0,0,100,26)
input_weight.x = input_height.x
input_weight.y = 305
input_weight.inputType = "numeric"
input_weight.size = 18
input_weight.text = "160"
input_weight.align = "center"
```

In order to make our text fields recognize when a number is entered, we have to add an event listener to the object. If you remember, an event is simply an action that happens within our app, and with an event listener, we are just waiting for an event to happen. In this case, we are waiting for the user to start typing into our input box. Once the event is triggered, we will call a function that will calculate the BMI.

The first step to creating the event listener is to add the function that is called when the event is triggered. Functions that work with event listeners are organized differently than a normal function. Let's take a look at an example function for an event listener:

```
function input_height:userInput(event)
  -- Enter event code here
end
```

The first difference you'll notice is the name of the function. The function name is the same as the name of the object we are creating the listener for. The function name also includes `:userInput`. The addition of `:userInput` to our function name is telling our function what type of event we are listening for. Finally, the function is accepting one argument—`event`. With the `event` variable, we can find out what phase the user is at with the input box. The event comes with four phases:

- **Began**: This phase represents when the user has tapped the input box for the first time.
- **Editing**: This phase represents when the user starts to type.
- **Ended**: This phase represents when the user has tapped another area outside of the input box.
- **Submitted**: This represents when the user has hit the return key on the keyboard.

If you would like to learn more about events, Corona has more information at `http://docs.coronalabs.com/api/type/Event.html`.

Now, let's add a functionality to our BMI calculator by adding two event listener functions to `view2.lua`. After the native user interface objects, add the following functions within the `enterScene()` function:

```
function input_height:userInput(event)
  if(event.phase == "ended" or
  event.phase == "submitted") then
user_bmi = (input_weight.text / (input_height.text *
  input_height.text)) * 703
```

```
txt_calculateBMI.text =
  math.floor( (user_bmi * 10) + 0.5) * 0.1
  end
end

function input_weight:userInput(event)
  if(event.phase == "ended" or
  event.phase == "submitted") then
user_bmi = (input_weight.text / (input_height.text *
   input_height.text)) * 703
txt_calculateBMI.text = math.floor( (user_bmi *
  10) + 0.5) * 0.1

end
end
```

Both functions operate the same way, except the first function is for the `input_height` user interface object and the second function is for the `input_weight` user interface object. These functions accept an argument called `event`, and when the user is in the ended or submitted phase, the function will calculate the BMI and update the text on the `txt_calculateBMI` display object.

The last piece required to get our calculator functional is to add the event listeners to the appropriate display objects. We can do this by adding `:addEventListener(eventType)` to the display object. Let's add the following lines after the event listener functions:

```
input_height:addEventListener("userInput")
input_weight:addEventListener("userInput")
```

If you've been following along, you can test the BMI calculator in our app by going to the **Your BMI** tab and entering your height and weight. After entering your height and weight, you will see your updated BMI under **Your BMI**.

> If you are on Windows, native text fields are not supported on the simulator, and you will need to publish to your device for full testing. We will cover publishing in *Appendix, Tools, Services, and Marketing*.

So far, the calculator works great. However, we need to add another element to make the user experience better. Most modern apps will allow you to touch an area of the background to exit the input process; we will put in this functionality by attaching an event listener to the background:

```
function rect_upperBackground:tap(e)
  native.setKeyboardFocus(nil)
end
```

```
rect_upperBackground:addEventListener("tap", rect_upperBackground)

function rect_background:tap(e)
  native.setKeyboardFocus(nil)
end

rect_background:addEventListener("tap", rect_background)
```

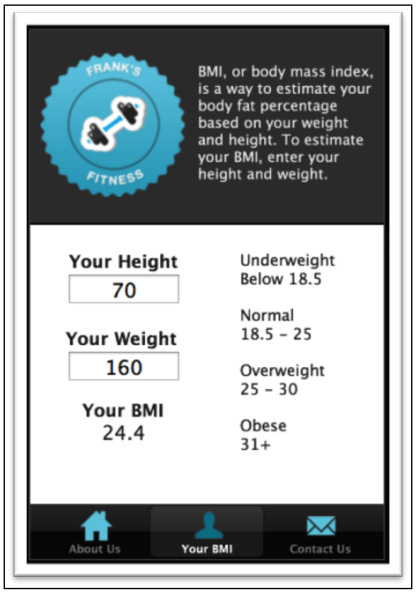
With this new functionality, we created an event listener for a tap event on the app's background. When the background is tapped, the keyboard focus is set to `nil`, which means hide the keyboard.

The final piece of functionality that we will add to `view2.lua` is removing the user interface objects when the user switches tabs. Inside the `exitScene()` function, we will add `display.remove(objectName)` to explicitly remove the input boxes. Add the following code inside the `exitScene()` function:

```
display.remove(input_height)
display.remove(input_weight)
```

By calling `display.remove()`, we are ensuring the user interface objects are removed from the screen.

If you've been following along, `view2.lua` will look like the following screenshot. The **Your BMI** tab provides information about the body mass index, and accepts input from users to calculate their BMI. In the next view, we will give users the ability to find the gym and send e-mail to it.



view3.lua – contact us

The third view will allow users to see the location of Frank's Fitness and give users a way to contact Frank's Fitness. When we created the project, Corona created `view1.lua` and `view2.lua`. Since we are missing `view3.lua`, we will have to create it. The easiest way to create `view3.lua` is to duplicate `view1.lua` and rename it to `view3.lua`. After you've duplicated the file, you should have three view files in your project folder—`view1.lua`, `view2.lua`, and `view3.lua`.

Let's open up `view3.lua` in your favorite text editor.

Since we duplicated `view1.lua`, our first step is to remove everything besides the variable local group inside our `createScene()` function. Your `createScene()` function should look like the following:

```
function scene:createScene( event )
  local group = self.view
end
```

Next, let's add some variables that will hold the content for `view3.lua`. Place the following variables after the local group variable:

```
contactMessage = "We have all the right tools to help
  you build a happier you. If you would like to schedule
  an appointment or if you just have some questions,
  please contact us!"
  contactAddress = "1625 SW 16th Indianapolis"
  contactPhone = "317-555-5555"
  contactEmailMsg = "Tap to send us an email at"
  contactEmail = "frank@franksfitnessgym.com"
```

After we've added the variables, we are going to add five text display objects and one image display object to our screen:

```
img_location = display.newImageRect( "location.png", 320, 175)
  img_location.x = display.contentCenterX
  img_location.y = 87
  group:insert( img_location )

txt_contact = display.newText( contactMessage, 0, 0, 300, 200, native.
systemFont, 14 )
  txt_contact:setTextColor( 22, 22, 22 )
  txt_contact.x = display.contentCenterX
  txt_contact.y = 300
  group:insert( txt_contact )
```

```
txt_contactAddress = display.newText( contactAddress, 0, 0,
  native.systemFontBold, 16 )
  txt_contactAddress:setTextColor( 22, 22, 22 )
  txt_contactAddress.x = display.contentCenterX
  txt_contactAddress.y = 295
  group:insert( txt_contactAddress )

txt_contactPhone = display.newText( contactPhone, 0, 0,
  native.systemFontBold, 16 )
  txt_contactPhone:setTextColor( 22, 22, 22 )
  txt_contactPhone.x = display.contentCenterX
  txt_contactPhone.y = 335
  group:insert( txt_contactPhone )

txt_contactEmailMsg = display.newText( contactEmailMsg, 0, 0,
  native.systemFontBold, 16 )
  txt_contactEmailMsg:setTextColor( 22, 22, 22 )
  txt_contactEmailMsg.x = display.contentCenterX
  txt_contactEmailMsg.y = 375
  group:insert( txt_contactEmailMsg )

txt_contactEmail = display.newText( contactEmail, 0, 0,
  native.systemFontBold, 16 )
  txt_contactEmail:setTextColor( 22, 22, 22 )
  txt_contactEmail.x = display.contentCenterX
  txt_contactEmail.y = 395
  group:insert( txt_contactEmail )
```

The last text display object prompts users to tap on the e-mail address to send an e-mail to Frank's Fitness. To build this functionality into our app, we need to add an event listener to the e-mail address.

Inside the function that is called by the event, we will use another feature of Corona's native user interface—`native.showPopup()`. This feature will display a pop-up window that allows users to send e-mails without leaving the app. Add the following code after the text display objects within the `createScene()` function to allow users to send e-mails from within the app:

```
function txt_contactEmail:tap(e)
  native.showPopup("mail",{
    to = "foo@bar.com",
    subject = "Contact from Frank's Fitness App",
```

```
        body = "Please send me more information
    about Frank's Fitness!"
        })
      end

    txt_contactEmail:addEventListener("tap", txt_contactEmail)
```

If you are on Windows, the pop up will not work, and you will have to build the device to test the e-mail functionality.

If you've been following along, your `view3.lua` file will look like the following screenshot. We've added display objects and provided an easy way for users to send an e-mail to Frank's Fitness. Next, we will be configuring our project to get the most out of our app.

config.lua – setting up dynamic content resolution

Just another one of the great features that the Corona SDK product provides is the ability to dynamically scale your content. With today's mobile devices, we are seeing more and more retina displays, and dynamic scaling is the mobile development solution. With dynamic scaling, you can easily provide higher-resolution images that will be substituted into your app when necessary.

If you've downloaded the starter kit, you've probably noticed that some images have a suffix, `@2x`. The `@2x` suffix stands for a higher-resolution image. For the Frank's Fitness, app the starter kit included `logo.png` and `logo@2x.png`. `logo.png` as the standard image, and `logo@2x.png` as the higher-resolution image.

To set up dynamic scaling, we have to open up `config.lua` in our favorite text editor. The new project that we've created will already include `config.lua`, and it's located in your project folder. Once you have it open, you will see the following code:

```
application = {
  content = {
    width = 320,
    height = 480,
    scale = "letterBox",
    fps = 30,

    --[[
    imageSuffix = {
    ["@2x"] = 2,
    }
    --]]
  },

  --[[
  -- Push notifications

  notification =
  {
    iphone =
    {
      types =
      {
        "badge", "sound", "alert", "newsstand"
      }
    }
  }
  --]]
}
```

Before we set up dynamic scaling, let's cover what's in the file. `config.lua` is one big table that contains different settings for your app. The table application contains two tables called `content` and `notification`. The table content has four properties enabled: `width`, `height`, `scale`, and `fps`—and one property disabled: `imageSuffix`. The disabled property is what we will use to set up dynamic scaling.

- `width`: This property sets the width of the app in pixels.
- `height`: This sets the height of the app in pixels.
- `scale`: This property tells our app how to treat screens of different sizes. Right now, we are using `letterbox`, which will create two black borders around the app when necessary. Other options for `scale` are `none`, `zoomEven`, and `zoomStretch`.
- `fps`: This property sets the frames per second. In most business apps, 30 frames per second is sufficient.
- `imageSuffix`: This property will set up the dynamic scaling for our app. By removing comments in line 8 and line 12 (the comment lines look like `--[[` and `]]--`), we will enable dynamic scaling. The property `imageSuffix` is set to the value `["@2x"] = 2`. This is telling our app to use images with the suffix `@2x` when the screen size is double the width and height.

For example, if you set your `config.lua` file to 320 x 480 and have enabled dynamic scaling, your app will automatically swap out images for higher-resolution images when on a screen that's 640 x 480. This is a perfect solution when you want your app to look great on the iPhone 3GS (320 x 480), the iPhone 4, and iPhone 4S (640 x 960). You can also add support for even higher resolution devices by adding `["@4x"] = 4` to your `config.lua` file.

If you would like to learn more about dynamic scaling, visit Corona Labs at `http://developer.coronalabs.com/content/configuring-projects`.

App icons

After we've created a functional app for Frank's Fitness, the last piece that we need to add is the application icon. The application icon is what shows up on the mobile device. In the starter kit, you will have access to a set of icons that can be dropped into your project folder. Here's where each icon will be used:

- `icon.png`: Used on iPhone/iPod/iPad
- `icon@2x.png`: Used on iPhone/iPad

- `icon-72.png`: Used on iPad
- `icon-ldpi.png`, `icon-mdpi.png`, `icon-hdpi.png`, and `icon-xhdpi.png`: Used on Android devices

Corona will use the appropriate icon based on the resolution of the device.

Summary

Congratulations! You've created your very first business app for a local gym. We've learned how to use Corona-specific features such as Storyboard, display objects, and widgets. We also learned how to add functionalities to help people easily calculate their BMI and to send an e-mail to Frank's Fitness.

A big misconception about the Corona SDK platform is that it's only meant for gaming applications. In this chapter, I hope you learned that Corona SDK is not just for games, but for great business apps too.

In the next chapter, we will learn how to create your first game with Corona SDK.

The image of the gym that was used in `view1.lua` of Frank's Fitness is courtesy of Luigi Diamanti from `http://www.freedigitalphotos.net`.

4
Creating a Puzzle App

In the previous chapter, we discussed how to use Corona's feature set to create our first business app. Although developing business style apps can be the cornerstone of your company, they can get a little boring. Inside this chapter, we will learn how to create our very first game with Corona SDK. Here are the topics that we will be covering in this chapter:

- How to drag-and-drop images
- Understanding events
- Using display groups
- Using the scroll view widget
- Using loops efficiently

Let's get started!

Since we've already covered a lot of the basics in previous chapters, we are going to move at a faster pace so that we can jump right into the game mechanics.

A quick introduction to puzzle games

Puzzle games are a genre of video games that have been around for decades. These types of games challenge players to use logic and critical thinking to complete patterns. There is a large variety of puzzle games available, and in this chapter, we'll start by learning how to create a 3-by-3 jigsaw puzzle titled *My Jigsaw Puzzle*.

In My Jigsaw Puzzle, players will have to complete a jigsaw puzzle by using nine puzzle pieces. Each puzzle piece will have an image on it, and the player will have to match the puzzle piece to the puzzle board by dragging the pieces from right to left. When the puzzle piece matches the correct location, the game will lock in the piece. Let's take a look at the final game product.

Downloading the starter kit

Before creating our puzzle app, you can get the starter kit for the jigsaw puzzle from the code files available with this book. The starter kit includes all of the graphics that we will be using in this chapter.

My Jigsaw Puzzle

For the Frank's Fitness app, we used Corona's built-in new project creator to help us with setting up our project. With My Jigsaw Puzzle, we will be creating the project from scratch. Although creating a project from scratch can be more time consuming, the process will introduce you to each element that goes into Corona's new project creator. Creating the project will include creating the `build.settings`, `config.lua`, `main.lua`, `menu.lua`, and `gameplay.lua` files. As done in previous chapters, we will be using Storyboard to manage the different scenes.

Before we can start creating the files for our project, we will need to create a new project folder on your computer. This folder will hold all of the files that will be used in our app.

build.settings

The first file that we will create is the `build.settings` file. This file will handle our device orientation and specify our icons for the iPhone. Inside our `build.settings` file, we will create one table named `settings`, which will hold two more tables named `orientation` and `iphone`.

The `orientation` table will tell our app to start in landscape mode and to only support `landscapeLeft` and `landscapeRight`. The `iphone` table will specify the icons that we want to use for our app.

To create the `build.settings` file, create a new file named `build.settings` in your project's folder and input the following code:

```
settings = {

  orientation =
  {
    default = "landscapeRight",
    supported = {
      "landscapeLeft",
      "landscapeRight" },
  },

  iphone =
  {
    plist =
    {
      CFBundleIconFile = "Icon.png",
      CFBundleIconFiles = {
         "Icon.png",
         "Icon@2x.png",
         "Icon-72.png",
      }
    }
  }
}
```

config.lua

Next, we will be creating a file named `config.lua` in our project's folder. The `config.lua` file is used to specify any runtime properties for our app. For My Jigsaw Puzzle, we will be specifying the width, height, and scale methods. We will be using the letterbox scale method, which will uniformly scale content as much as possible. When letterbox doesn't scale to the entire screen, our app will display black borders outside of the playable screen.

To create the `config.lua` file, create a new file named `config.lua` in your project's folder and input the following code:

```
application =
{
  content =
  {
    width = 320,
    height = 480,
    scale = "letterbox"
  }
}
```

main.lua

Now that we've configured our project, we will be creating the `main.lua` file—the start point for every app. For now, we are going to keep the file simple. Our `main.lua` file will hide the status bar while the app is active and redirect the app to the next file—`menu.lua`.

To create `main.lua`, create a new file named `main.lua` in your project's folder and copy the following code into the file:

```
display.setStatusBar ( display.HiddenStatusBar )
local storyboard = require ( "storyboard" )
storyboard.gotoScene("menu")
```

menu.lua

Our next step is to create the menu for our app. The menu will show a background, the game title, and a play button. The player can then tap on the **PLAY GAME** button to start playing the jigsaw puzzle.

To get started, create a new file in your project's folder called `menu.lua`. Once the file has been created, open `menu.lua` in your favorite text editor. Let's start the file by getting the widget and storyboard libraries. We'll also set up Storyboard by assigning the variable `scene` to `storyboard.newScene()`.

```
local widget = require "widget"
local storyboard = require( "storyboard" )
local scene = storyboard.newScene()
```

Next, we will set up our `createScene()` function. In *Chapter 3*, *Creating Your First App*, we learned that the function `createScene()` is called when entering a scene for the first time. Inside this function, we will create objects that will be displayed on the screen. Most of the following code should look familiar by now. Here, we are creating two image display objects and one widget. Each object will also be inserted into the variable group to let our app know that these objects belong to the scene `menu.lua`.

```
function scene:createScene( event )
  local group = self.view
  background = display.newImageRect(
  "woodboard.png", 480, 320 )
  background.x = display.contentWidth*0.5
  background.y = display.contentHeight*0.5
  group:insert(background)

  logo = display.newImageRect( "logo.png", 400, 54 )
  logo.x = display.contentWidth/2
  logo.y = 65
  group:insert(logo)
```

```
    function onPlayBtnRelease()
      storyboard.gotoScene("gameplay")
    end

    playBtn = widget.newButton{
      default="button-play.png",
      over="button-play.png",
      width=200, height=83,
      onRelease = onPlayBtnRelease
    }
    playBtn.x = display.contentWidth/2
    playBtn.y = display.contentHeight/2
    group:insert(playBtn)

  end
```

After the `createScene()` function, we will set up the `enterScene()` function. The `enterScene()` function is called after a scene has moved on to the screen. In My Jigsaw Puzzle, we will be using this function to remove the `gameplay` scene. We need to make sure we are removing the `gameplay` scene so that the jigsaw puzzle is reset and the player can play a new game.

```
function scene:enterScene( event )
  storyboard.removeScene( "gameplay" )
end
```

After we've created our `createScene()` and `enterScene()` functions, we need to set up our event listeners for Storyboard.

```
scene:addEventListener( "createScene", scene )
scene:addEventListener( "enterScene", scene )
```

Finally, we end our `menu.lua` file by adding the following line:

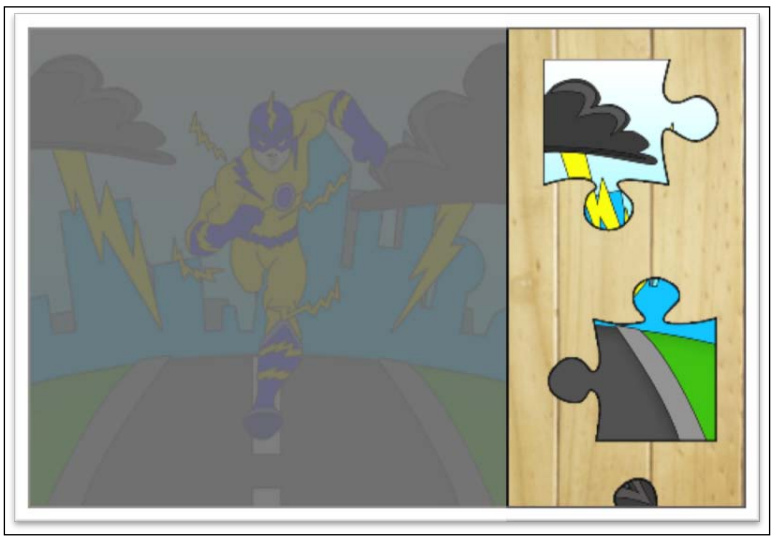
```
return scene
```

This line of code lets our app know that we are done with this scene.

Now that we've added the last line, we have finished editing `menu.lua`, and we will now start setting up our jigsaw puzzle.

gameplay.lua

By now, our game has been configured and we have set up two files—`main.lua` and `menu.lua`. In our next step, we will be creating the jigsaw puzzle. The following screenshot shows the puzzle that we will be making:



Getting local libraries

To get started, create a new file called `gameplay.lua` and open the file in your favorite text editor. Similar to our `menu.lua` file, we need to start the file by getting in other libraries and setting up Storyboard.

```
local widget = require("widget")
local storyboard = require( "storyboard" )
local scene = storyboard.newScene()
```

Creating variables

After our local libraries, we are going to create some variables to use in `gameplay.lua`. When you separate the variables from the rest of your code, the process of refining your app later becomes easier.

```
_W = display.contentWidth
_H = display.contentHeight
```

```
puzzlePiecesCompleted = 0
totalPuzzlePieces = 9
puzzlePieceWidth = 120
puzzlePieceHeight = 120
puzzlePieceStartingY = {
80,220,360,500,640,780,920,1060,1200 }
puzzlePieceSlideUp = 140
puzzleWidth, puzzleHeight = 320, 320
puzzlePieces = {}

puzzlePieceCheckpoint = {
  {x=-243,y=76},
  {x=-160, y=76},
  {x=-76, y=74},
  {x=-243,y=177},
  {x=-143, y=157},
  {x=-57, y=147},
  {x=-261,y=258},
  {x=-176,y=250},
  {x=-74,y=248}
}

puzzlePieceFinalPosition = {
  {x=77,y=75},
  {x=160, y=75},
  {x=244, y=75},
  {x=77,y=175},
  {x=179, y=158},
  {x=265, y=144},
  {x=58,y=258},
  {x=145,y=251},
  {x=248,y=247}
}
```

Here's a breakdown of what we will be using each variable for in our app:

- `_W` and `_H`: These variables capture the width and height of the screen. In our app, we have already specified the size of our app to be 480 x 320.
- `puzzlePiecesCompleted`: This variable is used to track the progress of the game by tracking the number of puzzle pieces completed.
- `totalPuzzlePieces`: This variable allows us to tell our app how many puzzle pieces we are using.

- `puzzlePieceWidth` and `puzzlePieceHeight`: These variables specify the width and height of our puzzle piece images within the app.
- `puzzlePieceStartingY`: This table contains the starting Y location of each puzzle piece. Since we can't have all nine puzzle pieces on screen at the same time, we are displaying the first two pieces and the other seven pieces are placed off the screen below the first two. We will be going over this in detail when we add the puzzle pieces.
- `puzzlePieceSlideUp`: After a puzzle piece is added, we will slide the puzzle pieces up; this variable sets the sliding distance.
- `puzzleWidth` and `puzzleHeight`: These variables specify the width and height of our puzzle board.
- `puzzlePieces`: This creates a table to hold our puzzle pieces once they are added to the board.
- `puzzlePieceCheckpoint`: This table sets up the checkpoints for each puzzle piece in x and y coordinates. When a puzzle piece is dragged to the checkpoint, it will be locked into position. When we add the checkpoint logic, we will learn more about this in greater detail.
- `puzzlePieceFinalPosition`: This table sets up the final puzzle location in x and y coordinates. This table is only used once the puzzle piece passes the checkpoint.

Creating display groups

After we have added our variables, we are going to create two display groups to hold our display objects. Display groups are simply a collection of display objects that allow us to manipulate multiple display objects at once. In our app, we will be creating two display groups—`playGameGroup` and `finishGameGroup`. `playGameGroup` will contain objects that are used when the game is being played and the `finishGameGroup` will contain objects that are used when the puzzle is complete. Insert the following code after the variables:

```
playGameGroup = display.newGroup()
finishGameGroup = display.newGroup()
```

The shuffle function

Our next task is to create a shuffle function for My Jigsaw Puzzle. This function will randomize the puzzle pieces that are presented on the right side of the screen. Without the shuffle function, our puzzle pieces would be presented in a 1, 2, 3 manner, while the shuffle function makes sure that the player has a new experience every time.

Creating the shuffle function

To create the shuffle function, we will start by creating a function named `shuffle`. This function will accept one argument (`t`) and proceed to randomize the table for us. We're going to be using some advanced topics in this function, but before we start explaining it, let's add the following code to `gameplay.lua` under our display group:

```
function shuffle(t)
  local n = #t
  while n > 2 do
    local k = math.random(n)
      t[n] = t[k]
      t[k] = t[n]
      n = n - 1
  end
  return t
end
```

At first glance, this function may look complex; however, the code gets a lot simpler once it's explained. Here's a line-by-line breakdown of our shuffle function.

The `local n = #t` line introduces two new features—`local` and `#`. By using the keyword `local` in front of our variable name, we are saying that this variable (`n`) is only needed for the duration of the function or loop that we are in. By using local variables, you are getting the most out of your memory resources and practicing good programming techniques. For more information about local variables, visit `www.lua.org/pil/4.2.html`. In this line, we are also using the `#` symbol. This symbol will tell us how many pieces or elements are in a table. In our app, our table will contain nine pieces or elements.

Inside the `while` loop, the very first line is `local k = math.random(n)`. This line is assigning a random number between `1` and the value of `n` (which is `9` in our app) to the local variable `k`. Then, we are randomizing the elements of the table by swapping the places of two pieces within our table. Finally, we are using `n = n - 1` to work our way backwards through all of the elements in the table.

Dragging-and-dropping display objects

Now we are going to set up the core functionality for our app—dragging-and-dropping display objects. To allow our players to drag-and-drop display objects, we have to set up a function that will allow them to grab an object and drag it around the screen. We can accomplish this by attaching an event listener to the object with the function that handles the dragging. However, before we can attach the event listener, we need to set up a function dedicated to this purpose.

A sample drag function with the associated object event listener would be as follows:

```
local function onDrag(event)
  local t = event.target

  if event.phase == "began" then

  elseif t.isFocus then
    if "moved" == event.phase then

    elseif "ended" == event.phase or
    "cancelled" == event.phase then

    end
  end

  return true
end
puzzlePiece:addEventListener("touch", onDrag)
```

In the example function, we can see the function named `onDrag` and the event listener on the object `puzzlePiece`. When the player touches the object `puzzlePiece`, the function `onDrag` is called.

Inside the function `onDrag`, the argument `event.target` is passed to the variable `t`. This line is assigning the touched object to the variable `t` so we can run some checks on the object later.

After the assignment, we are checking to see what phase the touch is in. The `began` phase is used when the player first touches the object. The `moved` phase is used when the player moves his/her finger on the screen to drag an object, and the phases `ended` or `cancelled` are are used when the player stops touching the screen or an unexpected event happens (such as an interruption from a phone call).

Before the `moved`, `ended`, or `cancelled` phases are checked, the function checks for `t.isFocus`. By using `t.isFocus`, our function is asking whether or not the player is touching the display object.

Finally, the last line inside the function is `return true`. This line of code is telling our app that the touch event was successful. Otherwise, your app will assume an error occurred, which may break its functionality.

Creating the drag function

Now that we understand the basics of the drag function, let's add the function `onDrag()` to our app below the shuffle function in `gameplay.lua`.

```
local function onDrag(event)
  local t = event.target

  if event.phase == "began" then


  elseif t.isFocus then
    if "moved" == event.phase then

    elseif "ended" == event.phase
    or "cancelled" == event.phase then

    end
  end
  return true
end
```

Our first addition to the `onDrag()` function is within the first `if-then` statement of `event.phase == "began"`. Within the first `if-then` statement, we will make the puzzle piece larger when it's selected, move the puzzle piece to the front, set the focus to the puzzle piece, and store the starting location of the puzzle piece.

```
t.xScale, t.yScale = 1.3, 1.3
t:toFront()
display.getCurrentStage():setFocus( t )
t.isFocus = true
xOrigin,yOrigin = t.x,t.y
t.x0 = event.x - t.x
t.y0 = event.y - t.y
```

If you remember, we assigned the variable `t` to `event.target`. This line stores the object within the variable and allows us to make changes to the object, which is the puzzle piece. So, you can think of `t` as the puzzle piece. Since we are introducing some new functionality, let's breakdown what's happening within the `began` phase:

- `t.xScale` and `t.yScale`: When we set the `xScale` and `yScale` properties of an object, we are making it bigger or larger. If `1` represents the original size, `1.3` represents a size that's 130 percent larger.
- `t:toFront()`: This ensures that the object stays on top of everything in the display hierarchy.

- `display.getCurrentStage():setFocus(t)`: With this line, we are setting the focus to the variable `t`, or the puzzle piece that was touched.
- `t.isFocus`: When we set this value to `true`, we are making sure that the object is the focus.
- `xOrigin` and `yOrigin`: We've set these variables to the initial location of the puzzle piece on the screen. We will be using these later to reset the piece in case the player wants to select another piece or there's an unexpected interruption.
- `t.x0` and `t.y0`: These variables are slightly different from the initial puzzle piece location because we are setting the values to the touch location minus the puzzle piece location. This is an important value because although the player can only move the puzzle piece, the player can touch anywhere on the piece to focus in on it. When we start to move the puzzle piece, we will use `t.x0` and `t.y0` to calculate movement.

Our second addition is within the `if-then` statement `"moved" == event.phase`. Within the `moved` phase, we will be moving the selected puzzle piece according to the touch location. We accomplish this by subtracting the touch location from the variables `t.x0` and `t.y0`.

```
t.x = event.x - t.x0
t.y = event.y - t.y0
```

Our third addition is within the `if-then` statement `"ended" == event.phase` or `"cancelled" == event.phase`. These phases will signify when the player has stopped touching the puzzle piece or an unexpected interruption has happened. The player can stop touching the puzzle piece by lifting their finger or dragging the piece into the correct location.

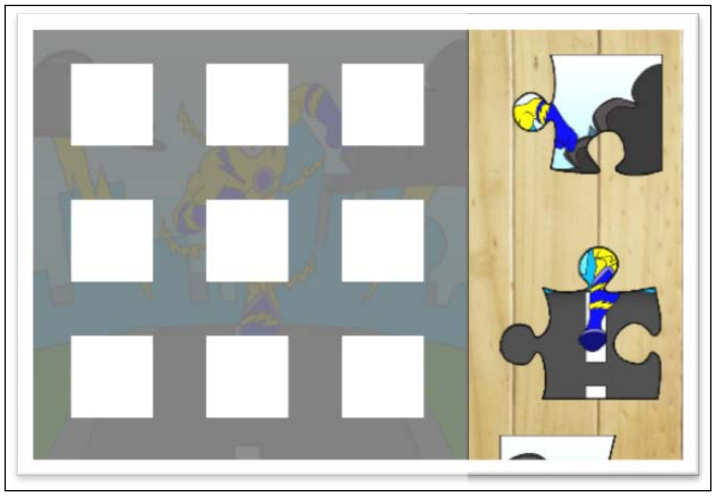
When the phase is `ended` or `cancelled`, we will remove the focus from the puzzle piece. By removing the focus from the puzzle piece, we allow the player to select another piece. Inside the block `elseif "ended == event.phase or "cancelled" == event.phase`, place the following lines of code:

```
display.getCurrentStage():setFocus( nil )
t.isFocus = false
```

When we created `gameplay.lua`, we created a table named `puzzlePieceCheckpoint` that stored the checkpoint location of each puzzle piece in x and y coordinates. This table allows us to perform a check to see if the player has dragged a puzzle piece to the correct location. If the puzzle piece is in the right location, we'll lock it. Otherwise, we'll reset the puzzle piece.

In order to perform this check, we are using a property of the puzzle piece object named `id`. The `id` object represents the location of the puzzle piece in relation to the board and will let us know which checkpoint to use. For example, if the `id` is equal to `1`, we know the piece belongs in the top-left corner, and the checkpoint is set to `x = -246` and `y = 76`. If the `id` object is equal to `9`, we know the piece belongs in the bottom-right corner, and the checkpoint is going to be `x = -74` and `y = 248`. The checkpoints that were created are relevant to the location of the puzzle piece and not to the top-left corner of the screen (which is 0,0).

To make our game easier, we have allowed players to drag the puzzle piece within 30 pixels of the puzzle piece. If we didn't allow players this ability, they would have to drag the piece to the exact checkpoint that we have specified. For a better visualization, take a look at the following screenshot:



Each white square represents a checkpoint for a puzzle piece. If the puzzle piece is dropped within the white square (our checkpoint), the puzzle piece will be locked into position and the player can move on to the next piece. Otherwise, the puzzle piece will be reset to the sidebar on the right. Now that we know how we are checking for the right location, let's add the following code right after the line `t.isFocus = true`:

```
if t.x <= puzzlePieceCheckpoint[t.id].x+30
  and t.x >= puzzlePieceCheckpoint[t.id].x-30
  and t.y <= puzzlePieceCheckpoint[t.id].y+30
  and t.y >= puzzlePieceCheckpoint[t.id].y-30 then

  -- Add Future Code Here
```

```
else
  t.xScale,t.yScale = 1,1
  t.x, t.y = xOrigin, yOrigin
end
```

After we've set up our checkpoint with an `if-then` statement, we are going to create the functionality for when the piece does match. We want to add the puzzle piece to the group `puzzleGroup`, move the piece behind the other puzzle pieces in the display hierarchy, set the piece to moved, position the piece according to the table `puzzlePieceFinalPosition`, and remove the ability to move the puzzle piece. Below the comment `-- Add Future Code Here`, add the following code:

```
puzzleGroup:insert(t)
t:toBack()
t.place = "moved"

t.x, t.y = puzzlePieceFinalPosition[t.id].x,
puzzlePieceFinalPosition[t.id].y

t:removeEventListener("touch",onDrag)
```

Next, we'll want to slide the rest of the puzzle pieces up the sidebar. This is a nice feature so the player doesn't have to go searching for the next puzzle piece. Right below `t:removeEventListener`, add the following lines of code:

```
for i = 1, local_sumPuzzle do
if puzzlePieces[i].place == "start"
and puzzlePieces[i].y > yOrigin then

puzzlePieces[i].y = puzzlePieces[i].y - puzzlePieceSlideUp
     end
end
```

The last addition to the checkpoint statement is to check whether the game is over or not. We'll check to see if the game is over by comparing the variable `puzzlePiecesComplete` with the total number of puzzle pieces. If the game is over, we will hide the display group `playGameGroup` and show the display group `finishGameGroup`. Add the following lines of code after the previous `for` statement:

```
puzzlePiecesCompleted = puzzlePiecesCompleted+1
if puzzlePiecesCompleted == totalPuzzlePieces then
playGameGroup.isVisible = false
finishGameGroup.isVisible = true
end
```

Now that we've set up the drag functionality for My Jigsaw Puzzle, we'll be creating our `createScene()` function inside `gameplay.lua`. Inside this function, we'll create a new display group and two image display objects.

```
function scene:createScene( event )
  local group = self.view

  puzzleGroup = display.newGroup()

  woodBoard = display.newImageRect("woodboard.png",
  480, 320)
  woodBoard.x = 562
  woodBoard.y = _H/2
  woodBoard:toBack();
  puzzleGroup:insert(woodBoard)

  puzzleBackground = display.newImageRect(
  "puzzle-base.png",
  puzzleWidth, puzzleHeight)
  puzzleBackground.x = 160
  puzzleBackground.y = _H/2
  puzzleBackground.alpha = .15
  puzzleGroup:insert(puzzleBackground)
end
```

Next, we'll add a new widget, called scroll view, to our game. Since not all of our puzzle pieces will be available at once, the scroll view widget will allow players to scroll through the puzzle pieces. Just like the other widgets we've added, we have the ability to set the viewable height, viewable width, and position. We also have the ability to set the total height and width of the scroll view. Add the following widget inside the `createScene` function, but after the line `puzzleGroup:insert(puzzleBackground)`:

```
scrollView = widget.newScrollView{
  top=0,
  left = _W - 160,
  height=_H,
  width = 160,
  hideBackground = true,
  scrollWidth = 50 ,
  scrollHeight = 1000 }
```

At the beginning of `gameplay.lua`, we created a function that will shuffle the elements of a table. Before we add our puzzle pieces to the screen, we are going to shuffle them so that they will show up in a random order. Insert the following line after the `scrollView` widget:

```
local shufflePuzzleY = shuffle(puzzlePieceStartingY)
```

After our puzzle pieces table has been shuffled, we are going to use a `for` loop to add the puzzle pieces to the screen. Each puzzle piece will be positioned, given an ID, given a touch event listener (to allow players to drag the puzzle piece), and inserted into the scroll view widget.

```
  for i = 1, totalPuzzlePieces do
  puzzlePieces[i] = display.newImageRect
  (i..".png", puzzlePieceWidth, puzzlePieceHeight)
  puzzlePieces[i].x = 85
  puzzlePieces[i].y = shufflePuzzleY[i]
  puzzlePieces[i].id = i
  puzzlePieces[i].place = "start"
  puzzlePieces[i]:addEventListener("touch", onDrag)
  scrollView:insert(puzzlePieces[i])
end
```

We are going to add the `scrollView` group (the group containing our puzzle pieces) into the `puzzleGroup` display group and then insert it into `playGameGroup`. We've created two different display groups to allow us to manipulate our display objects easier. The `puzzleGroup` display group contains objects that will not be interacted with directly, and the `playGameGroup` contains all the objects that are shown when the game is active. Insert the following lines after the previous `for` loop:

```
puzzleGroup:insert(scrollView)
playGameGroup:insert(puzzleGroup)
```

After we've created the display groups, we need to create what happens when the puzzle is complete. To create the `finishGame` elements, we will create two text display objects and one image display object. The text display object, `txt_return`, will allow users to return to `menu.lua`. Add the following lines of code after the line `playGameGroup:insert(puzzleGroup)`:

```
function returnToMenu(event)
  if(event.phase == "ended") then
    storyboard.gotoScene("menu")
  end
end
```

```
txt_gameComplete = display.newText(
"Puzzle Complete", 0,0,120,100,native.systemFont,22)
txt_gameComplete.x = 400
txt_gameComplete.y = _H/2 - 50
finishGameGroup:insert(txt_gameComplete)

txt_return = display.newText(
"Return to Menu",
0,0,120,100,native.systemFont,22)
txt_return.x, txt_return.y = 400, _H/2 + 50
txt_return:addEventListener("touch",
returnToMenu)
finishGameGroup:insert(txt_return)

baseBG = display.newImageRect("puzzle-base.png",
puzzleWidth, puzzleHeight)
baseBG.x, baseBG.y = 160, _H/2
finishGameGroup:insert(baseBG)
```

Next, we are going to add the two display groups, `finishGameGroup` and `playGameGroup`, to our Storyboard group. This will let our app know what objects belong to the current scene. After the line `finishGameGroup:insert (baseBG)`, insert the following lines:

```
group:insert(finishGameGroup)
group:insert(playGameGroup)
```

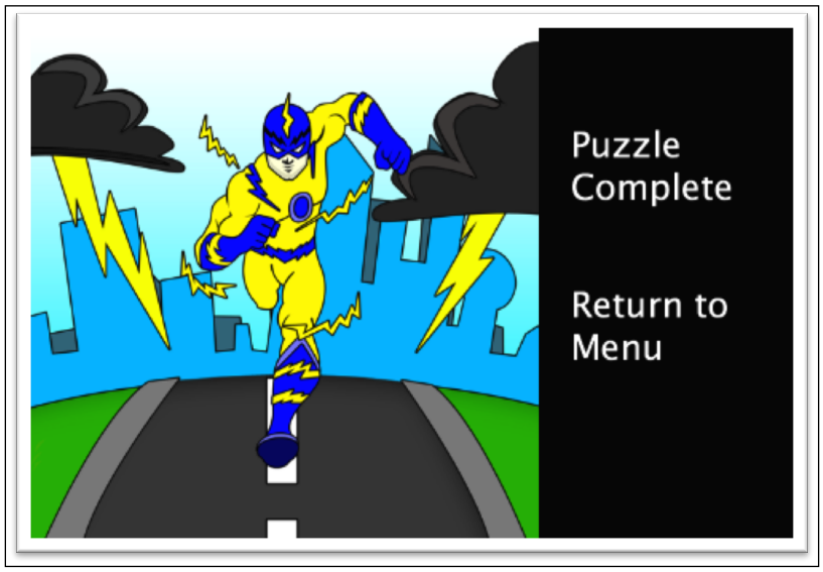
We are going to finish the `createScene()` function by hiding everything that belongs to `finishGameGroup` and showing everything that's in `playGameGroup`. Remember, `finishGameGroup` contains everything that will be shown when the game is done, and `playGameGroup` contains everything that is shown when the game is being played.

```
finishGameGroup.isVisible = false
playGameGroup.isVisible = true
```

Our final addition to `gameplay.lua` is to add the scene listeners to make Storyboard work for us.

```
function scene:enterScene( event ) end
function scene:exitScene( event ) end
function scene:destroyScene( event ) end

scene:addEventListener( "createScene", scene )
scene:addEventListener( "enterScene", scene )
scene:addEventListener( "exitScene", scene )
scene:addEventListener( "destroyScene", scene )

return scene
```

After you've added the scene listeners, you will have finished `gameplay.lua` and you will have completed My Jigsaw Puzzle. You can test your brand new game by starting Corona Simulator and opening up `main.lua`.



Summary

After reading this chapter, you will have a complete game that is ready to be played by you and your friends. After creating this game, you should now know how to drag-and-drop display objects, use display groups, use events, and use the scroll view widget. In My Jigsaw Puzzle, we only provided one puzzle for the player, and although it's a great puzzle, I suggest adding more puzzles to make the game more appealing to more players.

This chapter is an important milestone in your mobile app development path. We've already learned how to create a business app and a game using Corona SDK. In the next chapter, we will discover how to publish our apps to iTunes and Google Play.

The image of the wood board that was used in this app is a courtesy of Scott Chan from `www.freedigitalphotos.net`. If you plan on using this image in your app, you must attribute the work to Scott Chan according to the terms of use at `www.freedigitalphotos.net`.

5
Testing and Publishing

Now that you have finished your app-creating journey, it's time to invite others to enjoy your work. Within this chapter, you will learn what it takes to get your app in the App Store. This chapter is meant as a guide in your publishing process, and we'll be learning about the overall process instead of a detailed step-by-step process. We'll also learn about other learning materials, third-party tools, and the community around Corona SDK.

We will be discussing the following topics in this chapter:

- How to test your app
- Publishing to iTunes
- Publishing to Google Play
- Other resources and the Corona community

How to test your app

Once you've finished your app, it's strongly suggested to test your app on an actual device to find any bugs. The process to test your app on a device for iOS or Android is very different. However, building for both devices can be done through the Corona Simulator found under **File | Build**.

iOS

In order to test for a device on iOS, you will need a provisioning profile and a Mac. A provisioning profile contains your name, app ID, and a distribution certificate. You can create a provisioning profile by creating an account at `http://developer.apple.com/` for $99 per year. Once you have created an account, you will be able to create provisioning profiles for testing purposes.

Android

For Android, you will need a key to build and test your apps. This key is a certificate that is held by the developer that identifies the author of an app. Corona provides a default key that you can use, however, the default key will install a pop up in your app notifying that you need a private key. You can obtain a private key by creating an account at `http://developer.android.com/` for $25.

For the full set of instructions and details about testing your app on iOS and Android, I strongly suggest visiting the following links:

- **iOS**: `http://developer.coronalabs.com/content/building-devices-iphoneipad`
- **Android**: `http://developer.coronalabs.com/content/building-devices-android`

Publishing your app

Once your app is built, you can publish your app so that it's available for download. The following sections discuss the steps to publish your app to Apple's App Store and Google Play.

Publishing to Apple's App Store

With your finished app, getting your app into the App Store is simple. To get started, you will need a developer account and to create a new application through iTunes Connect. When you create a new application, you will have to upload your app description, keywords, icons, screenshots, and other assets. Once your app is submitted, it will undergo Apple's review process to ensure your app meets their submission criteria. Although Apple lists their submission criteria within your developer account, the most common reasons for rejection are if your app crashes, steals user information, or it is controversial. Corona Labs provides a detailed list of the publishing process at `http://developer.coronalabs.com/content/building-devices-iphoneipad`.

Publishing to Google Play

Publishing your app to Google Play is a lot simpler than publishing to Apple's App Store. Once you have a developers account, visit `https://play.google.com/apps/publish` and add a new application to your account. Once you add a new application, you will have to upload your app description, icons, screenshot, and other assets. After you have entered the required information, you can publish your app almost instantly to Google Play. Google does not have a review process, but they will remove your app if they find that your app violates their content polices. For example, apps that are controversial, offensive, or dangerous are removed.

Corona SDK also provides the option to build and publish your app to the Amazon App Store (`developer.amazon.com`) and the Nook App Store (`nookdeveloper.barnesandnoble.com`). The process to publish your app on both stores is very similar to the processes listed earlier.

Other resources and the Corona community

Over the years, the community surrounding Corona SDK has grown by leaps and bounds. There are several resources available to developers, and I've listed some of these resources here:

- **Corona forums**: If you want to get involved with the Corona community or get answers to your app development questions, visit `http://developer.coronalabs.com/forum/`. On the forum, feel free to ask any questions about advertising, development, graphics, and anything else you can think of.
- **Corona Geek**: Some great folks at Corona Labs host a weekly Google+ hangout to discuss latest news, marketing tips, and more. To learn more about the weekly hangout, visit `www.CoronaGeek.com`.
- **Tutorials**: Outside of this book, you can find plenty of tutorials regarding Corona SDK. Visit `http://www.coronalabs.com/resources/tutorials/` or `http://thatssopanda.com/corona-sdk-tutorials/` for more learning materials.

Thank you for reading *Corona SDK Application Design*. Hopefully you will have learned what it takes to create a business or game app with Corona SDK. Don't forget, you have a wealth of knowledge available to you at `www.coronalabs.com`. Corona SDK, code less play more!

Tools, Services, and Marketing

While developing apps with Corona SDK, I have discovered some great tools that help speed up development even further, and I have discovered some great and easy-to-do marketing tips that every developer should consider. In this appendix, I'll be discussing some of the tools that I've come across and I'll share some of the marketing tips that I've learned. Here's an overview of what I will be discussing with you:

- Third-party tools
- Third-party services
- App marketing tips

Third-party tools

Third-party tools for Corona SDK have been built to help speed up your development by providing a development environment, file management, or even the ability to edit physics shapes.

- **Outlaw**: This Corona project manager allows you to manage your project easily and more effectively. This tool also offers more features such as an autocomplete function. Find out more about it at `http://outlawgametools.com/`.

- **Texture Packer**: If you are going to be developing games with Corona SDK, you will eventually use sprites in your games. Texture Packer can be a lifesaver, because it can create a sprite sheet for your app in just a few seconds. If you would like to find out more about sprites and sprite sheets, you can visit `docs.coronalabs.com/api/library/display/newSprite.html`, or for more information about Texture Packer, visit `www.codeandweb.com/texturepacker`.
- **Physics Editor**: Similar to Texture Packer, Physics Editor is a tool built to speed up your game development. This tool will allow you to create custom collision shapes for an even better game. Find out more about this tool at `www.codeandweb.com/physicseditor`.

Third-party services

Third-party services are separate from third-party tools because services are ongoing tools to help you market or optimize your app. Here are some of the services that I use to help market my apps:

- **Search Man SEO**: Unless you are pouring hundreds of dollars into promoting your app, most people will find your app by searching. Search Man SEO is a service that will show how your app stacks up against other apps in the various markets and what keywords your competitors are using. Find out more about Search Man SEO at `www.searchman.com`.
- **AppCodes**: Similar to Search Man SEO, AppCodes will help you track your competitors and will help you discover what keywords your competitors are using. Find out more about AppCodes at `www.appcod.es`.

Marketing tips

The marketing tips discussed in this section are not meant to discuss every marketing avenue you can use as an app developer. Instead, here I have revealed what has worked for me in my own mobile adventure.

Get your friends and family to use your app. As the developer, you know the ins and outs of your apps and you know how everything works together. However, other people may not understand how your app works, and the best way to find out is to have your friends and family use your app.

After you release your app, get your family and friends to review and rate it on the App Store or Google Play. Once your app is in the wild, your friends and family may want to support you, and the best way for them to support you is to ask them to leave a review for you app.

When creating icons, avoid using words. I've seen some icons, even one of my own, with words used in the app icon. Instead of using words, you should use an identifiable graphic in your icon. For example, our game My Jigsaw Puzzle uses a puzzle piece as the icon instead of the game title.

Create great screenshots. Screenshots of your app are arguably the top selling feature of your app in the App Store or Google Play. When people are considering whether or not to purchase your app, they will look at the screenshots, and some of the best screenshots that I've seen are of actual gameplay and tell a story about the app. Your screenshots should intrigue the person enough to buy it.

www.allitebooks.com

Index

A

addNumbers() function 20
Amazon App Store
URL 75
Android
about 6
app, testing for 74
app
publishing 74
testing 73
AppCodes
about 78
URL 78
Apple's App Store
app, publishing to 74
app, publishing
to Apple's App Store 74
to Google Play 75
app, testing
for Android 74
for iOS 73
arithmetic expressions, Lua 13
automatic scene purging 28

B

began phase, event variable 43
build.settings file, Franks Fitness App 25
build.settings file, My Jigsaw Puzzle
creating 55

C

comparison operators, Lua 13
config.lua file, Franks Fitness App 25, 49
config.lua file, My Jigsaw Puzzle
creating 56
Corona Cider
URL 23
Corona community 75
Corona Complete
URL 23
Corona forums
about 75
URL 75
Corona Geek
about 75
URL 75
Corona Labs
about 6
URL, for publishing process 74
Corona SDK
about 5
My Jigsaw Puzzle, creating 53
prerequisites 6
resources 75
third-party services 78
third-party tools 77
URL, for downloading 7
using 8-10
Corona SDK, installing
for Mac OS X 7, 8
for Microsoft Windows 8
Corona Simulator 8, 73
Corona-specific editors
Corona Cider 23
Corona Complete 23
Outlaw IDE 23
Corona Terminal 9
createScene() function 57
createScene function, Franks Fitness App 27

D

debugger 9
destroyScene function, Franks Fitness App 28
display.getCurrentStage():setFocus(t) 65
display groups, My Jigsaw Puzzle
finishGameGroup 61
playGameGroup 61
display.newRect() function 32
display objects, Franks Fitness App 32
display objects, My Jigsaw Puzzle
dragging-and-dropping 62, 63
distanceTraveled variable 16
drag function, My Jigsaw Puzzle
creating 64-70

E

editing phase, event variable 43
ended phase, event variable 43
enterScene() function 58
enterScene function, Franks Fitness App 27
event variable
began phase 43
editing phase 43
ended phase 43
submitted phase 43
exitScene() function 45
exitScene function, Franks Fitness App 28

F

feetTable variable 19
finishGameGroup, My Jigsaw Puzzle 61
for loop 16
Franks Fitness App
application icon, creating 50
build.settings 25
config.lua file 25, 49
creating 21-25
dynamic content resolution, setting up 49, 50
icon1.png 26
icon2.png 26
main.lua 26-35
scenes, destroying 28
scenes, switching 28
Storyboard, visualizing 26, 27
text editor, using 22
view1.lua 26
view2.lua 26
widgets, using 29, 30
functions, Lua
using 19

G

gameplay.lua file, My Jigsaw Puzzle
creating 59
display groups, creating 61
local libraries 59
shuffle function 61
shuffle function, creating 62
variables, creating 59, 60
gameTimer() function 20
getCurrentSceneName() function 34
Google Play
app, publishing to 75

I

icon1.png, Franks Fitness App 26
icon2.png, Franks Fitness App 26
icon, Franks Fitness App
creating 50
if-then-else statement 15
if-then statement
about 14
understanding 14
input_height user interface object 44
installation, Corona SDK
for Mac OS X 7, 8
for Microsoft Windows 8
iOS
about 6
app, testing for 73

K

Kindle 6

L

loops, Lua
about 15
for loop 16
while loop 17
Lua
about 5
arithmetic expressions 13
comparison operators 13
decision making 14
functions, using 19
loops 15
tables 18
terminal, using 12
URL 12
variables, using with 11

M

Mac OS X
Corona SDK, installing for 7, 8
main.lua file, My Jigsaw Puzzle
creating 56
main.lua, Franks Fitness App
about 26-34
view1.lua 35
view2.lua 38
view3.lua 46
marketing tips 78, 79
menu.lua file, My Jigsaw Puzzle
creating 56-58
metersTable variable 19
Microsoft Windows
Corona SDK, installing for 8
My Jigsaw Puzzle
build.settings file, creating 55
config.lua file, creating 56
creating 54
gameplay.lua file, creating 59
main.lua file, creating 56
menu.lua file, creating 56, 57, 58
starter kit, downloading 54

N

Nook 6
Nook App Store
URL 75
Notepad++
about 23
URL 23

O

onDrag function 63
onFirstView() funtion 32
Outlaw
about 77
URL 77
Outlaw IDE
URL 23

P

Physics Editor
about 78
URL 78
playerStepsTaken variable 16
playGameGroup, My Jigsaw Puzzle 61
print() command 13
Puzzle App
creating 53
display objects, dragging-and-dropping 62
My Jigsaw Puzzle 54
puzzle games 53
puzzle games 53
puzzlePiece object 63

R

rect_upperBackground*toBack() 33

S

scenes, Franks Fitness App
destroying 28
switching 28

Search Man SEO
about 78
URL 78
shuffle function, My Jigsaw Puzzle
about 61
creating 62
sprite sheets
URL 78
Starter 6
Storyboard, Franks Fitness App
createScene function 27
destroyScene function 28
enterScene function 27
exitScene function 28
visualizing 26, 27
storyboard.newScene() 57
string 12
Sublime Text 23
submitted phase, event variable 43

T

tables, Lua 17, 18
terminal
using 12
text editor
about 22
Corona-specific editors 23
Notepad++ 23
Sublime Text 23
Texture Packer
about 78
URL 78
Text Wrangler 23
third-party services, Corona SDK
about 78
AppCodes 78
Search Man SEO 78
third-party tools, Corona SDK
about 77
Physics Editor 78
Texture Packer 78
t.isFocus 65
toBack() function 33
t:toFront() 64
t.x0 variable 65
t.xScale 64
txt_calculateBMI display object 44
t.y0 variable 65
t.yScale 64

V

variables, using with Lua
about 11
Boolean value 12
rules 12
string 12
view1.lua, Franks Fitness App 26-38
view2.lua, Franks Fitness App 26-45
view3.lua, Franks Fitness App 46, 47

W

while loop 17
widgets
using, in Franks Fitness App 29, 30

X

xOrigin 65

Y

yOrigin 65

[PACKT] PUBLISHING

Thank you for buying Corona SDK Application Design

About Packt Publishing

Packt, pronounced 'packed', published its first book "*Mastering phpMyAdmin for Effective MySQL Management*" in April 2004 and subsequently continued to specialize in publishing highly focused books on specific technologies and solutions.

Our books and publications share the experiences of your fellow IT professionals in adapting and customizing today's systems, applications, and frameworks. Our solution based books give you the knowledge and power to customize the software and technologies you're using to get the job done. Packt books are more specific and less general than the IT books you have seen in the past. Our unique business model allows us to bring you more focused information, giving you more of what you need to know, and less of what you don't.

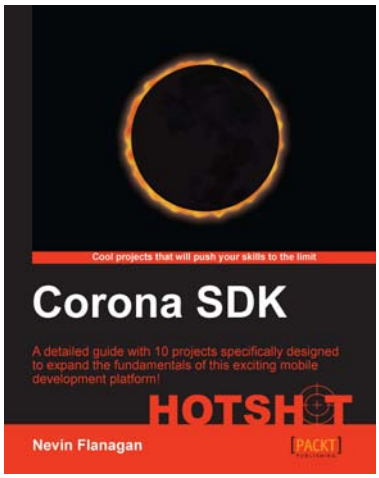
Packt is a modern, yet unique publishing company, which focuses on producing quality, cutting-edge books for communities of developers, administrators, and newbies alike. For more information, please visit our website: `www.packtpub.com`.

Writing for Packt

We welcome all inquiries from people who are interested in authoring. Book proposals should be sent to `author@packtpub.com`. If your book idea is still at an early stage and you would like to discuss it first before writing a formal book proposal, contact us; one of our commissioning editors will get in touch with you.

We're not just looking for published authors; if you have strong technical skills but no writing experience, our experienced editors can help you develop a writing career, or simply get some additional reward for your expertise.

[PACKT]
PUBLISHING

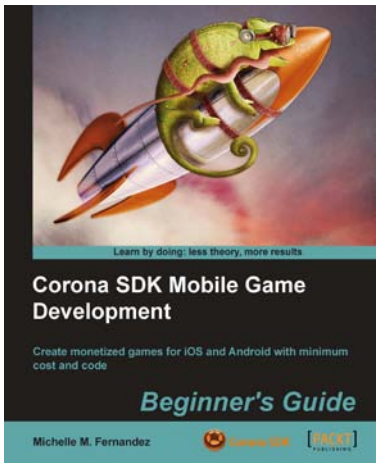


Corona SDK Hotshot

ISBN: 978-1-84969-430-8 Paperback: 358 pages

A detailed guide with 10 projects specifically designed to expand the fundamentals of this exciting mobile development platf orm!

1. Ten fully developed code projects that build on previous projects and present new techniques.
2. Freely reusable art and sound files included with every project help you jumpstart your own development.
3. Numerous advanced techniques to make the most out of Corona's features and the Lua programming language.



Corona SDK Mobile Game Development: Beginner's Guide

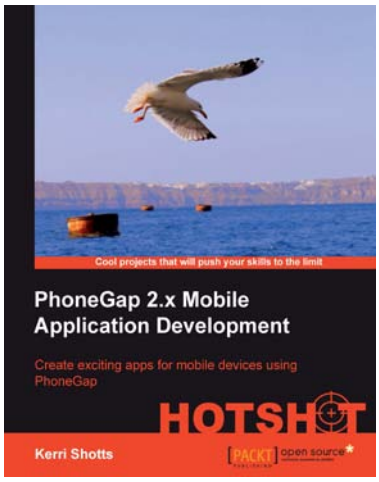
ISBN: 978-1-84969-188-8 Paperback: 408 pages

Create monetized games for iOS and Android with minimum cost and code

1. Build once and deploy your games to both iOS and Android
2. Create commercially successful games by applying several monetization techniques and tools
3. Create three fun games and integrate them with social networks such as Twitter and Facebook

Please check **www.PacktPub.com** for information on our titles

[PACKT]
PUBLISHING



PhoneGap 2.x Mobile Application Development Hotshot

ISBN: 978-1-84951-940-3 Paperback: 388 pages

Create exciting apps for mobile devices using PhoneGap

1. Ten apps included to help you get started on your very own exciting mobile app
2. These apps include working with localization, social networks, geolocation, as well as the camera, audio, video, plugins, and more
3. Apps cover the spectrum from productivity apps, educational apps, all the way to entertainment and games

---

Android 3.0 Application Development Cookbook

ISBN: 978-1-84951-294-7 Paperback: 272 pages

Over 70 working recipes covering every aspect of Android development

1. Written for Android 3.0 but also applicable to lower versions
2. Quickly develop applications that take advantage of the very latest mobile technologies, including web apps, sensors, and touch screens
3. Part of Packt's Cookbook series: Discover tips and tricks for varied and imaginative uses of the latest Android features

.

Please check **www.PacktPub.com** for information on our titles

www.allitebooks.com